UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE MATEMATICHE, FISICHE E NATURALI



CORSO DI LAUREA IN
INFORMATICA MAGISTRALE

TESI DI LAUREA IN
ALGORITMI E STRUTTURE DATI

TITOLO
IL PROBLEMA DEL NEIGHBORHOOD SEARCH SU GRAFI TEMPORALI

Relatore:
Prof.
Gualà Luciano
Correlatore:
Dott. Straziota Alessandro

Candidato:
Salvucci Franco
Matricola: *0368692*

Anno Accademico 2025/2026

Indice

1 Introduzione **5**

2 Neighborhood Search su Grafi Statici **9**
2.1 Definizione del problema 9
2.2 Soluzione esatta e suoi limiti 10
2.2.1 Locality Sensitive Hashing 11
2.3 Neighborhood search mediante LSH 16

3 Neighborhood Search su Grafi Temporali **17**
3.1 Definizioni preliminari 17
3.2 Varianti del problema 18
3.3 Il ruolo degli sketch 19

4 Variante 1: NS con intervalli ad ampiezza fissa **21**
4.1 Algoritmo Brute Force 21
4.2 Algoritmo di generazione dell'indice LSH 22
4.3 Algoritmo di risposta alla query 23
4.4 Analisi della complessità 24
4.5 Oracolo $\mathcal{O}$ per il recupero degli sketch 30
4.5.1 Range tree 31
4.5.1.1 Range Query 32
4.5.2 Definizione della struttura dati 33
4.5.3 Query sulla struttura dati 34
4.5.3.1 Caso speciale di query: tempo $O(1)$ con 1-MinHash . 34
4.5.4 Analisi dei costi 36

5 Variante 2: NS con intervalli ad ampiezza variabile **38**
5.1 Dal problema originale al sottoproblema 38
5.2 Rappresentazione del primo blocco tramite rettangoli 39
5.3 Algoritmo di generazione dell'indice 42
5.4 Algoritmo di risposta alla query 45
5.5 Analisi della complessità 47

6 Esperimenti **48**
6.1 Valutazione sperimentale dell'oracolo $\mathcal{O}$ per il recupero degli sketch . 48
6.1.1 Configurazione sperimentale 48
6.1.2 Strategie a confronto 49
6.1.3 Risultati e discussione 49
6.2 Valutazione sperimentale: Neighborhood Search con finestre di ampiezza fissa 50
6.2.1 Configurazione sperimentale 50
6.2.2 Pruning temporale 50

6.2.3 Prestazioni e memoria ausiliaria 51
6.2.4 Completezza delle risposte rispetto alla Brute Force 52
6.2.5 Sintesi dei risultati 54
6.3 Valutazione sperimentale del filtro preliminare dell'oracolo $\mathcal{O}$ 54

7 Conclusioni e Sviluppi Futuri **57**
7.1 Conclusioni 57
7.2 Sviluppi futuri 58

Bibliografia **60**

Elenco delle figure

1 Confronto delle curve ad S al variare della suddivisione in bande (b) e righe (r) per una dimensione fissa di $k = b \cdot r = 100$ funzioni hash. . . 14
2 Esempio di grafo temporale $\mathcal{G} = (V, E, \lambda)$. 18
3 Confronto tra il vicinato temporale istantaneo $N^4(v)$ e quello sull'intervallo $I = [2, 4]$, ottenuto come unione dei vicinati istantanei. 19
4 Analisi della dimensione di $W(v)$,utilizzando valori di test $T = 14, \mu = 2, T - \mu = 12$ 27
5 Confronto tra un Range Tree standard, e l'oracolo $\mathcal{O}$. 31
6 Rappresentazione grafica di $X_{\mathcal{R}}(\mu)$. 41
7 Rappresentazione grafica di $X_{\mathcal{R}_v}(\mu) \cap X_{\mathcal{R}_u}(\mu)$. 42
8 Esempio RangeTree 4D. 43
9 Tempo medio di recupero dello sketch al variare di μ: confronto fra l'oracolo $\mathcal{O}$ e la ricostruzione naïve. (a) Grafo Erdős–Rényi con $\delta = 32$, $k = 128$; (b) grafo sintetico standard con $\delta = 200$, $k = 64$; (c) grafo sintetico standard con $\delta = 350$, $k = 128$. 49
10 Analisi della cardinalità di $W(v)$ e del fattore di guadagno del pruning temporale per i dataset `SFHH-conf-sensor` (a) e `ia-facebook-wall-wosn-dir` (b). 51
11 Tempi medi di query, memoria ausiliaria e costruzione: BF contro LSH+W (in alto, scale logaritmiche) e confronto tra LSH+W, LSH–W e Alt+W (in basso, scale lineari). 53
12 Risposte positive cumulative e confronto finale fra BF e LSH+W: chess (in alto) ed Enron (in basso). 54
13 Numero di nodi candidati distinti restituiti dal filtro preliminare dell'oracolo $\mathcal{O}$ al variare dell'ampiezza temporale μ. Risultati sui dataset: (a) `CollegeMsg`, (b) `SFHH-conf-sensor`, (c) `ia-facebook-wall-wosn-dir`, (d) `ia-enron-email-dynamic` ed (e) `chess_year`. 56

Elenco delle tabelle

1 Parametri temporali e risultati del pruning per tutti i dataset considerati. Il guadagno rappresenta la media dei fattori $H_\mu/|W(v)|$ riportati nei grafici. 51
2 Tempo medio di query (μs): BF e tre strategie LSH. 52
3 Costruzione (s) e memoria ausiliaria (MiB) degli indici LSH. Per BF, costruzione e memoria specifiche sono trascurabili; l'oracolo condiviso è escluso per tutte le strategie. 52
4 Completezza delle risposte positive della LSH rispetto alla BF 53
5 Dataset, numero di sorgenti interrogate e ampiezze temporali considerate (espresse in unità discretizzate). 55

1 Introduzione

I grafi costituiscono uno strumento naturale per rappresentare sistemi in cui le relazioni tra le entità rivestono un ruolo centrale, come le reti sociali, le reti di comunicazione e i sistemi di interazione tra utenti. In molti di questi contesti, un problema fondamentale consiste nell'individuare nodi che presentano comportamenti o caratteristiche strutturali simili. Una possibile nozione di similarità è quella basata sui *vicinati*: due nodi sono considerati simili quando condividono una parte significativa dei propri vicini. Questo tipo di ricerca, noto come *Neighborhood Search*, può essere formulato come un problema di ricerca per similarità tra insiemi. Dato un nodo di riferimento, l'obiettivo è individuare un altro nodo il cui vicinato sia sufficientemente simile a quello del primo, secondo una misura di similarità opportunamente definita. In questa tesi viene considerata la similarità di Jaccard, che quantifica il rapporto tra la cardinalità dell'intersezione e quella dell'unione di due insiemi.
Nei grafi statici, il vicinato di un nodo è un insieme fissato. Tuttavia, in molte applicazioni reali le relazioni evolvono nel tempo: gli utenti di una rete sociale interagiscono in momenti differenti, le comunicazioni tra due soggetti si verificano in determinati periodi e le connessioni di una rete possono essere attive soltanto in specifici istanti. In questi scenari, rappresentare tutte le interazioni mediante un unico grafo statico comporta una perdita di informazione temporale. In particolare, due nodi potrebbero avere vicinati molto simili durante un determinato periodo, pur risultando poco simili quando si considerano tutte le interazioni dell'intera rete. Questa osservazione motiva lo studio del *Neighborhood Search su grafi temporali*, oggetto della presente tesi. Consideriamo un grafo temporale $G = (V, E, \lambda)$, in cui a ogni arco $e \in E$ è associato un insieme di istanti temporali $\lambda(e)$. Per un nodo v e un intervallo I, indichiamo con $N^I(v)$ l'insieme dei nodi che hanno interagito con v almeno una volta durante I. Fissate una soglia di similarità τ e un'ampiezza μ, il problema consiste nel preprocessare il grafo in modo da poter rispondere alla seguente interrogazione: dato un nodo v, individuare, se esistono, un nodo $u \neq v$ e un istante iniziale t tali che

$$J\big(N^{[t,t+\mu]}(v), N^{[t,t+\mu]}(u)\big) \geq \tau.$$

In altre parole, vogliamo individuare due nodi i cui vicinati siano sufficientemente simili **in almeno una finestra temporale della durata considerata**. L'istante iniziale della finestra non è noto a priori e deve essere individuato durante la ricerca. La dimensione temporale introduce difficoltà algoritmiche che non emergono nel caso statico. Una soluzione esaustiva dovrebbe considerare tutte le finestre temporali ammissibili e, per ciascuna di esse, confrontare il vicinato del nodo interrogato con quello degli altri nodi del grafo. Il numero di confronti può quindi crescere rapidamente sia con il numero dei nodi sia con l'estensione dell'orizzonte temporale. Inoltre, i vicinati cambiano al variare della finestra: non è dunque sufficiente costruire una singola rappresentazione del vicinato di ogni nodo, come avviene nel caso statico. Per affrontare queste difficoltà, la tesi considera due varianti del problema, distinte dalle informazioni disponibili durante il preprocessing. Nella prima variante, l'ampiezza μ

è fissata prima della costruzione dell'indice e rimane costante per tutte le interrogazioni. Nella seconda variante, invece, μ viene specificata insieme al nodo interrogato, e la struttura dati deve quindi supportare finestre di ampiezza arbitraria. Sebbene le due formulazioni condividano lo stesso obiettivo, la diversa conoscenza dei parametri richiede strategie di indicizzazione differenti.
L'approccio adottato si basa su tecniche di *sketching* e *Locality Sensitive Hashing* (LSH). In particolare, gli sketch MinHash permettono di rappresentare un insieme mediante una firma compatta, preservando informazioni sulla sua similarità di Jaccard con altri insiemi. LSH sfrutta queste rappresentazioni per selezionare un insieme di nodi candidati, evitando di confrontare il nodo interrogato con tutti i nodi del grafo. La ricerca segue quindi uno schema *filter-and-verify*: una prima fase individua i candidati, mentre una seconda fase ne valuta la similarità mediante gli sketch. L'utilizzo di queste tecniche introduce un compromesso tra efficienza e accuratezza. Il filtraggio probabilistico può infatti escludere nodi che soddisfano la condizione di similarità, mentre il confronto mediante sketch fornisce una stima, e non necessariamente il valore esatto, della similarità di Jaccard. Le soluzioni studiate devono pertanto essere interpretate come algoritmi di ricerca approssimata.

Contributi della tesi. Un primo contributo, comune a entrambe le varianti, è la progettazione di una struttura dati per il recupero efficiente degli sketch dei vicinati temporali. La proprietà fondamentale utilizzata è la *componibilità* degli sketch MinHash: lo sketch dell'unione di più insiemi può essere ottenuto combinando gli sketch dei singoli insiemi. Sfruttando questa proprietà, viene costruito un oracolo temporale $\mathcal{O}$ basato su un albero di ricerca sugli istanti di attività di ciascun nodo. La struttura permette di recuperare lo sketch di un vicinato su un intervallo arbitrario senza ricostruire esplicitamente l'insieme dei vicini. Per sketch di dimensione k, il tempo di interrogazione è $O(k \log |\Lambda(v)|)$, dove $\Lambda(v)$ è l'insieme degli istanti di attività del nodo v. Lo spazio occupato è lineare nel numero di archi temporali, assumendo fissata la dimensione dello sketch. Nel caso particolare di una sola componente MinHash, il problema può inoltre essere ricondotto a una *Range Minimum Query*, ottenendo interrogazioni in tempo costante.
Per la **variante ad ampiezza fissa**, viene proposta una soluzione basata su un indice LSH che organizza congiuntamente le informazioni relative ai nodi e alle finestre temporali. L'idea consiste nell'estendere le chiavi delle tabelle LSH includendo, oltre alla firma del vicinato, l'istante iniziale della finestra. In questo modo, le collisioni vengono utilizzate per individuare nodi con vicinati potenzialmente simili nello stesso intervallo temporale. L'indicizzazione è inoltre accompagnata da una procedura di *pruning temporale*, che permette di ignorare le finestre nelle quali il nodo non presenta alcuna attività. Viene mostrato che il numero di finestre rilevanti dipende non soltanto dal numero di eventi associati a un nodo, ma anche dalla loro distribuzione temporale. La soluzione è accompagnata da un'analisi dei costi di preprocessing, spazio e interrogazione.

La **variante ad ampiezza variabile** presenta una difficoltà ulteriore: non conoscendo μ durante il preprocessing, costruire un indice LSH indipendente per ogni possibile ampiezza richiederebbe di considerare un numero quadratico di intervalli temporali. Per evitare questa enumerazione, viene studiata una diversa strategia di selezione dei candidati, basata sulla prima componente dello sketch MinHash. L'osservazione centrale è che, per ogni nodo, gli intervalli temporali possono essere rappresentati geometricamente mediante una famiglia di rettangoli sui quali tale componente assume un valore costante. Questa rappresentazione permette di trasformare la ricerca di nodi che condividono il medesimo valore MinHash in una ricerca ortogonale multidimensionale, risolvibile mediante un *Range Tree* a quattro livelli. Il filtro così ottenuto consente di individuare coppie nodo-finestra candidate senza costruire esplicitamente un indice per ogni possibile ampiezza. I candidati possono successivamente essere verificati utilizzando gli sketch completi restituiti dall'oracolo temporale $\mathcal{O}$. Anche questa soluzione è accompagnata da un'analisi teorica dei costi, pur mantenendo una natura euristica dovuta al filtraggio sulla singola componente MinHash.

Valutazione sperimentale. Le soluzioni proposte vengono studiate mediante esperimenti su grafi sintetici e cinque dataset temporali reali. La valutazione si concentra su tre aspetti: l'efficienza dell'oracolo per il recupero degli sketch, le prestazioni dell'algoritmo per intervalli ad ampiezza fissa e la selettività del filtro preliminare per intervalli ad ampiezza variabile. I risultati mostrano che l'oracolo temporale permette di recuperare gli sketch in tempi dell'ordine dei microsecondi, a fronte di tempi dell'ordine dei millisecondi, o superiori, richiesti dalla ricostruzione diretta. Per la variante ad ampiezza fissa, l'indicizzazione LSH e il pruning temporale consentono riduzioni significative dei tempi di interrogazione rispetto alla ricerca esaustiva, con fattori di accelerazione che, nei dataset considerati, variano da circa 200 a oltre 8000. Il vantaggio dipende tuttavia dalle caratteristiche del grafo e comporta un costo aggiuntivo di preprocessing e memoria. Inoltre, il filtraggio LSH può compromettere la completezza delle risposte: la recall rispetto alla ricerca esaustiva sugli stessi sketch varia sensibilmente tra i dataset, dal 12.17% al 98.07%. Per la variante ad ampiezza variabile, gli esperimenti valutano la capacità del filtro preliminare di restringere l'insieme dei nodi candidati. I risultati indicano che tale insieme può rimanere relativamente contenuto anche su grafi con decine di migliaia di nodi. La valutazione sperimentale non comprende tuttavia l'intera procedura di verifica dei candidati, e non permette quindi di trarre conclusioni definitive sulle prestazioni complessive della seconda soluzione. Nel complesso, la tesi studia come combinare tecniche di ricerca probabilistica per similarità e strutture dati per intervalli temporali, con l'obiettivo di rendere più efficiente la ricerca di vicinati simili in grafi temporali. L'analisi teorica e la valutazione sperimentale evidenziano sia i vantaggi delle soluzioni proposte sia i compromessi tra tempo di interrogazione, occupazione di memoria e completezza delle risposte.

Organizzazione della tesi. Il Capitolo 2 introduce il problema del Neighborhood Search su grafi statici e richiama le tecniche MinHash e LSH. Il Capitolo 3 presenta il modello di grafo temporale e formalizza le due varianti del problema. Il Capitolo 4 descrive la soluzione per intervalli ad ampiezza fissa, insieme all'oracolo per il recupero degli sketch temporali e alle relative analisi di complessità. Il Capitolo 5 affronta la variante ad ampiezza variabile, introducendo la rappresentazione geometrica degli intervalli e la struttura dati per la ricerca dei candidati. Il Capitolo 6 presenta infine la valutazione sperimentale delle soluzioni studiate.

2 Neighborhood Search su Grafi Statici

I grafi modellano sistemi in cui l'informazione risiede nelle relazioni tra le entità: reti sociali, grafi del web, grafi utenti-prodotti. In questi contesti un'operazione ricorrente è quella di individuare dei nodi "vicini" a un dato nodo target, dove la vicinanza può essere intesa in senso strutturale, come adiacenza, oppure in senso di similarità tra le connessioni dei nodi. Questo capitolo affronta il secondo caso, noto come *Neighborhood Search*, nel contesto dei grafi statici. Dopo aver formalizzato il problema (Sezione 2.1) e analizzato i limiti della soluzione esatta (Sezione 2.2), si introduce il *Locality Sensitive Hashing* (LSH, Sezione 2.2.1) e si descrive la soluzione che ne deriva (Sezione 2.3).

2.1 Definizione del problema

Un grafo statico è una coppia $G = (V, E)$, dove V è l'insieme degli n nodi ed E l'insieme degli m archi. Il *vicinato* $N(u)$ di un nodo u è l'insieme dei nodi adiacenti a u, e la sua cardinalità è detta grado di u, indicato con $deg(u)$. Il problema del Neighborhood Search è un problema di ricerca per similarità. Il problema di ricerca per similarità consiste nel trovare, all'interno di un grande insieme di dati, gli elementi più "vicini" o "somiglianti" a un elemento di riferimento (detto query), anziché cercare una corrispondenza esatta. Nel nostro caso, il problema può essere riscritto come: dato un nodo sottoinsieme di nodi $S \subseteq V$, individuare un nodo $v \in V$ il cui vicinato è simile a quello di S. Per quantificare la somiglianza tra vicinati si adotta una funzione di similarità

$$\sigma : 2^V \times 2^V \to [0, 1],$$

dove 2^V indica l'insieme delle parti di V. Tale funzione σ possiamo definirla, senza perdità di generalità, come

$$\sigma(A, B), \qquad \text{per ogni } A, B \subseteq V.$$

Il problema del Neighborhood Search può dunque essere enunciato come segue.

Definizione 2.1.1 (Neighborhood Search su grafi statici)**.** *Dati un grafo G, un sottoinsieme di nodi $S \subseteq V$ e una soglia $\tau \in [0, 1]$, restituire un nodo $v \in V$ tale che*

$$\sigma(S, N(v)) \geq \tau.$$

La scelta di usare la Jaccard Similarity come nostra funzione σ non è casuale. Essa infatti è una misura ampiamente utilizzata nell'ambito del *data mining su grafi*, trovando applicazione in problemi quali rilevamento di comunità, clustering di grafi e predizione di collegamenti fra nodi [1, 2, 3]. In particolare, la Jaccard Similarity permette di quantificare il grado di sovrapposizione tra due insiemi tenendo conto delle loro dimensioni. Per comprendere l'importanza di questa proprietà, si consideri

una misura di similarità più semplice, basata esclusivamente sulla cardinalità dell'intersezione tra due insiemi A e B, ossia $|A \cap B|$. Questa misura, pur quantificando il numero di elementi condivisi, non tiene conto delle dimensioni complessive degli insiemi e può quindi produrre risultati poco rappresentativi del loro effettivo grado di similarità. Un'altra misura nota in letteratura, che normalizza l'intersezione, è il *containment score*. Questa misura può essere definita come

$$\sigma_C(A, B) = \frac{|A \cap B|}{|A|}.$$

Il containment score esprime la frazione degli elementi di A contenuti anche in B, ma presenta lo svantaggio di non essere simmetrica. In generale, infatti, $\sigma_C(A, B) \neq \sigma_C(B, A)$. In particolare, se $B \subseteq A$ e $|B| \ll |A|$, si ottiene $\sigma_C(A, B) \ll 1$, mentre $\sigma_C(B, A) = 1$. Sebbene questa proprietà sia utile nei problemi di ricerca per contenimento, risulta meno adatta quando l'obiettivo è misurare la similarità reciproca tra due vicinati. La scelta naturale ricade quindi nel normalizzare la cardinalità dell'intersezione rispetto a quella dell'unione dei due insiemi, ottenendo la *similarità di Jaccard*:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Questa misura è simmetrica, assume valori nell'intervallo $[0, 1]$ ed è pari a 1 quando i due insiemi non vuoti coincidono e a 0 quando sono disgiunti. La normalizzazione rispetto all'unione permette di valutare la sovrapposizione in relazione alla dimensione complessiva degli insiemi, evitando di favorire sistematicamente quelli di cardinalità maggiore. L'utilizzo della similarità di Jaccard consente infatti di confrontare i vicinati considerando la proporzione di vicini condivisi, anziché il loro numero assoluto. Tale misura di similarità trova inoltre applicazione in numerosi altri ambiti, tra cui l'individuazione di documenti duplicati o quasi duplicati, il clustering e l'analisi di dati biologici ed ecologici.

2.2 Soluzione esatta e suoi limiti

La soluzione diretta confronta il vicinato di S con quello di ogni altro nodo, calcolando $\sigma(S, N(v))$ fermandosi non appena si supera la soglia. Il risultato è esatto, ma il costo per singola query è pari a $O(n\,|S|)$, e dunque, nel caso peggiore, *quadratico* nella dimensione del grafo. Risulta quindi necessario ridurre il numero di confronti, passando da una scansione completa a uno schema *filter-and-verify*: un filtro economico seleziona un piccolo insieme di candidati, sui quali la similarità viene poi verificata. In letteratura esiste un framework apposito per i task di *filter-and-verify*, chiamato **Locality Sensitive Hashing (LSH)**. Questo framework implementa lo schema filter-and-verify utilizzando delle funzioni hash specifiche, chiamate **funzioni hash locality-sensitive**.

2.2.1 Locality Sensitive Hashing

Una funzione di hash tradizionale è progettata per disperdere gli input in modo uniforme, così che anche differenze minime producano valori distinti. Il LSH adotta il principio opposto: una famiglia di funzioni di hash è detta *locality sensitive* se la probabilità di collisione è elevata per coppie di oggetti simili e bassa per coppie di oggetti dissimili. Più formalmente, possiamo fornire la seguente definizione [4].

Definizione 2.2.1 (Funzione Hash (d_1, d_2, p_1, p_2)-Locality Sensitive)**.** *Sia U un universo finito di elementi e sia $d : U \times U \to \mathbb{R}^+$ una metrica di distanza. Siano $o \leq p_2 < p_1 \leq 1$ e $d_1 \leq d_2$. Una famiglia di funzioni hash $\mathcal{H}$ è detta* (d_1, d_2, p_1, p_2)-Locality Sensitive *se, scegliendo uniformemente a caso h da $\mathcal{H}$, per ogni $x, y \in U$ abbiamo che:*

- *se $d(x, y) \leq d_1$, allora* $\Pr[h(x) = h(y)] \geq p_1$*;*
- *se $d(x, y) \geq d_2$, allora* $\Pr[h(x) = h(y)] \leq p_2$*.*

Ritorniamo ora al problema del Neighborhood Search in esame. Poiché la funzione di similarità adottata è l'indice di Jaccard, possiamo definire la corrispondente distanza come

$$d_J(A, B) = 1 - J(A, B).$$

Pertanto, due insiemi soddisfano la condizione di similarità $J(A, B) \geq \tau$ se e solo se $d_J(A, B) \leq 1 - \tau$. Idealmente, vorremmo disporre di una famiglia di funzioni hash che permetta di distinguere perfettamente gli insiemi simili da quelli dissimili. In particolare, fissata la soglia $t = 1 - \tau$, vorremmo che

$$\Pr[h(A) = h(B)] = \begin{cases} 1 & \text{se } d_J(A, B) \leq t, \\ 0 & \text{se } d_J(A, B) > t. \end{cases}$$

Questo rappresenta il comportamento ideale di una famiglia $(t, t, 1, 0)$-sensitiva. Disponendo di una tale funzione, la soluzione del Neighborhood Search sarebbe immediata. Durante il preprocessing, per ogni nodo $v \in V$ si calcolerebbe $h(N(v))$ e lo si inserirebbe nel bucket corrispondente di una tabella hash H:

$$H[h(N(v))] \leftarrow H[h(N(v))] \cup \{v\}.$$

Al momento della query, dato l'insieme S, sarebbe sufficiente calcolare $h(S)$ e accedere al bucket $H[h(S)]$. Qualsiasi nodo ammissibile contenuto in tale bucket soddisferebbe la condizione $J(S, N(v)) \geq \tau$, senza necessità di ulteriori verifiche. Una funzione hash con queste proprietà, tuttavia, non è generalmente realizzabile. Il problema deriva dal fatto che l'uguaglianza dei valori hash è una relazione transitiva, mentre la relazione di similarità definita da una soglia di Jaccard non lo è. Consideriamo, ad esempio, i tre insiemi

$$A = \{1\}, \qquad B = \{1, 2\}, \qquad C = \{2\}.$$

Vale che

$$J(A,B) = J(B,C) = \frac{1}{2}, \qquad J(A,C) = 0.$$

La funzione ideale dovrebbe quindi imporre $h(A) = h(B)$ e $h(B) = h(C)$, ma allo stesso tempo richiedere $h(A) \neq h(C)$, ottenendo una contraddizione. Non è dunque possibile, in generale, suddividere gli insiemi in bucket che corrispondano esattamente alle classi di similarità desiderate. L'approccio LSH supera questa difficoltà rinunciando a una separazione perfetta e utilizzando invece funzioni hash probabilistiche, nelle quali gli insiemi più simili hanno una maggiore probabilità di collidere.

Costruzione dell'indice LSH mediante bande. Supponiamo di disporre di una famiglia $\mathcal{H}$ di funzioni hash (d_1, d_2, p_1, p_2)-sensitiva. Campioniamo indipendentemente k funzioni dalla famiglia:

$$h_1, \ldots, h_k \overset{\text{i.i.d.}}{\sim} \text{Unif}(\mathcal{H}).$$

A ogni insieme A associamo una firma di lunghezza k, definita come

$$\mathcal{S}_k(A) = \langle h_1(A), h_2(A), \ldots, h_k(A) \rangle.$$

La tecnica delle bande, o *banding technique*, consiste nel suddividere questa firma in b blocchi, detti *bande*, ciascuno contenente r valori, in modo che $k = b \cdot r$. Per ogni $i \in \{1, \ldots, b\}$, definiamo la i-esima banda come

$$B_i(A) = \left\langle h_{r(i-1)+1}(A), \ldots, h_{ri}(A) \right\rangle.$$

Durante il preprocessing vengono costruite b tabelle hash $H_1, \ldots, H_b$. Per ogni nodo $v \in V$ e per ciascuna banda i, il nodo viene inserito nel bucket identificato dalla banda del suo vicinato:

$$H_i[B_i(N(v))] \leftarrow H_i[B_i(N(v))] \cup \{v\}.$$

Al momento della query, dato un insieme S, si calcola la sua firma e si interrogano i b bucket corrispondenti. L'insieme dei nodi candidati è quindi

$$C(S) = \bigcup_{i=1}^{b} H_i[B_i(S)].$$

Un nodo diventa candidato se condivide con S almeno una banda completa. Soltanto i nodi appartenenti a $C(S)$ vengono sottoposti alla successiva verifica di similarità. In questo modo non è necessario confrontare S con tutti i vicinati del grafo: il costo della ricerca dipende principalmente dal numero di candidati restituiti dai bucket.

Costruzioni AND e OR. La tecnica delle bande combina due costruzioni probabilistiche complementari, denominate *AND construction* e *OR construction*. [5]

- **AND construction.** Due insiemi collidono in una banda soltanto se coincidono tutti gli r valori hash che la compongono. Indicando con $s = \Pr[h(A) = h(B)]$ la probabilità di collisione di una singola funzione hash, l'indipendenza delle funzioni campionate implica che la probabilità di collisione in una banda sia s^r. Poiché s^r diminuisce all'aumentare di r per $0 < s < 1$, questa costruzione rende il filtro più selettivo, riducendo le collisioni tra insiemi poco simili. Tuttavia, aumenta anche la probabilità che due insiemi effettivamente simili non coincidano nella banda.

- **OR construction.** Due insiemi diventano candidati se collidono in almeno una delle b bande. La probabilità di non collidere in una singola banda è $1 - s^r$. Poiché le bande utilizzano funzioni hash indipendenti, la probabilità di non collidere in nessuna delle b bande è $(1 - s^r)^b$. La probabilità di diventare candidati è quindi
$$P(s) = 1 - (1 - s^r)^b.$$
Aumentare il numero di bande incrementa la probabilità di individuare coppie simili, riducendo il rischio di falsi negativi, ma aumentando anche il numero di candidati da verificare.

Prendendo $\mathcal{E}_1$ l'evento '$d(x, y) \leq d_1$' e $\mathcal{E}_2$ l'evento '$d(x, y) \geq d_2$', e sfruttando il fatto che $\mathcal{H}$ è una famiglia (d_1, d_2, p_1, p_2)-sensitiva, possiamo affermare che:

$$P(\mathcal{E}_1) \geq 1 - (1 - p_1^r)^b, \qquad P(\mathcal{E}_2) \leq (1 - p_2^r)^b.$$

Le due costruzioni permettono quindi di controllare il compromesso tra selettività e completezza del filtro. Una configurazione particolarmente selettiva riduce i falsi positivi, ossia i nodi dissimili che vengono comunque selezionati come candidati, ma può introdurre un maggior numero di falsi negativi, cioè nodi simili che non superano il filtro. I falsi positivi possono essere scartati mediante una successiva verifica esatta della similarità. I falsi negativi, invece, non vengono esaminati e non possono essere recuperati dalla fase di verifica.

Curvatura della funzione ed andamento ad "S". La funzione $P(s)$ presenta, per le configurazioni usuali dei parametri, un andamento simile a una curva ad "S". Per valori bassi di s, la probabilità di candidatura rimane contenuta; superata una regione di transizione, essa cresce rapidamente fino ad avvicinarsi a 1. Tale andamento può essere interpretato come un'approssimazione probabilistica della funzione hash ideale introdotta in precedenza: anziché separare nettamente gli insiemi simili da quelli dissimili, il filtro realizza una transizione graduale tra basse e alte probabilità di collisione. La regione di transizione è approssimativamente individuata dal valore

$$t \approx \left(\frac{1}{b}\right)^{1/r}. \tag{1}$$

Lo schema *AND/OR*, da cui deriva l'andamento ad "S" ed il punto di flesso $t \approx \left(\frac{1}{b}\right)^{1/r}$ sono stati introdotti ed analizzati qui [6].

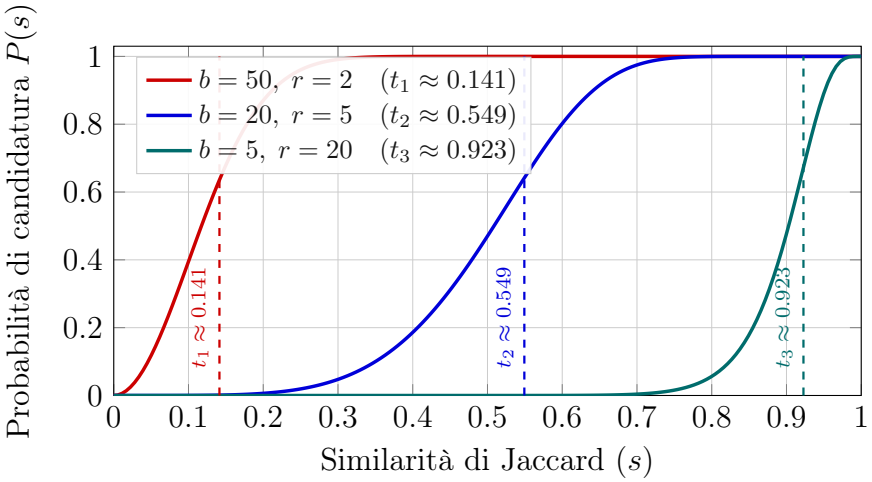


Figura 1: Confronto delle curve ad S al variare della suddivisione in bande (b) e righe (r) per una dimensione fissa di $k = b \cdot r = 100$ funzioni hash.

La Figura 1 mostra l'andamento della probabilità di candidatura per tre configurazioni dei parametri, mantenendo costante la lunghezza della firma $k = 100$. La configurazione con molte bande di piccola dimensione ($b = 50$, $r = 2$) è più permissiva: anche insiemi con similarità relativamente bassa possono diventare candidati. Al contrario, utilizzando poche bande di grandi dimensioni ($b = 5$, $r = 20$), la probabilità di collisione rimane bassa fino a valori elevati di similarità. La configurazione intermedia ($b = 20$, $r = 5$) rappresenta un compromesso tra questi due comportamenti. La scelta di b e r permette quindi di adattare il filtro alla soglia di similarità τ richiesta dal problema. Tale scelta deve tuttavia considerare che anche coppie con similarità superiore a τ possono non collidere in alcuna banda: la tecnica LSH non garantisce pertanto il recupero di tutte le coppie simili, ma consente di controllarne probabilisticamente la probabilità di candidatura.

LSH per similarità di Jaccard. Una famiglia di funzioni hash sensitiva rispetto alla distanza di Jaccard può essere costruita utilizzando la tecnica *MinHash* [7]. Sia U un universo finito di elementi e sia $\pi : U \to \{1, \ldots, |U|\}$ una permutazione di U. Dato un insieme non vuoto $A \subseteq U$, definiamo il suo valore MinHash rispetto alla permutazione π come

$$\mathrm{MH}(A, \pi) = \min_{x \in A} \pi(x).$$

Il MinHash associa quindi a ciascun insieme un singolo valore, corrispondente al minimo rango assunto dai suoi elementi nella permutazione considerata. La proprietà

fondamentale del MinHash è che, scegliendo π uniformemente a caso tra tutte le permutazioni di U, la probabilità che due insiemi non vuoti A e B abbiano lo stesso valore MinHash coincide esattamente con la loro similarità di Jaccard:

$$\Pr_{\pi}\left[\mathrm{MH}(A,\pi) = \mathrm{MH}(B,\pi)\right] = \frac{|A \cap B|}{|A \cup B|} = J(A,B). \tag{2}$$

Per comprendere questa proprietà, osserviamo che i due valori MinHash coincidono se e solo se l'elemento di rango minimo nell'unione $A \cup B$ appartiene all'intersezione $A \cap B$. Poiché la permutazione è scelta uniformemente a caso, ciascun elemento dell'unione ha la stessa probabilità di assumere il rango minimo. La probabilità di collisione è quindi pari al rapporto tra la cardinalità dell'intersezione e quella dell'unione. Questa proprietà mostra che MinHash definisce una famiglia di funzioni hash *locality sensitive*. Infatti, considerando la distanza di Jaccard

$$d_J(A,B) = 1 - J(A,B),$$

la probabilità di collisione diventa

$$\Pr_{\pi}[\mathrm{MH}(A,\pi) = \mathrm{MH}(B,\pi)] = 1 - d_J(A,B).$$

Pertanto, per $0 \leq d_1 < d_2 \leq 1$, la famiglia MinHash è $(1-d_1, 1-d_2, d_1, d_2)$-sensitiva: insiemi più vicini hanno una maggiore probabilità di collidere.

Costruzione dello sketch k-MinHash. Un singolo valore MinHash non è sufficiente per ottenere una stima affidabile della similarità tra due insiemi. Per questo motivo, si considerano k permutazioni $\pi_1, \ldots, \pi_k$, campionate indipendentemente e uniformemente dall'insieme di tutte le permutazioni di U. Lo sketch *k-MinHash* di un insieme A è definito come la concatenazione dei k valori MinHash:

$$\mathcal{S}_k(A) = \left(\mathrm{MH}(A,\pi_1), \ldots, \mathrm{MH}(A,\pi_k)\right). \tag{3}$$

A differenza del singolo MinHash, che restituisce un unico valore, il k-MinHash rappresenta quindi un insieme mediante una firma di lunghezza k.

Componibilità degli sketch MinHash. Un'ulteriore proprietà fondamentale del MinHash è la *componibilità* rispetto all'unione degli insiemi [8]. Dati due insiemi A e B, il valore MinHash della loro unione può essere ottenuto direttamente dai rispettivi valori MinHash, senza accedere agli elementi originali. Infatti,

$$\begin{aligned}
\mathrm{MH}(A \cup B, \pi) &= \min_{x \in A \cup B} \pi(x) \\
&= \min\left\{ \min_{x \in A} \pi(x), \min_{x \in B} \pi(x) \right\} \\
&= \min\left\{ \mathrm{MH}(A,\pi), \mathrm{MH}(B,\pi) \right\}.
\end{aligned}$$

Questa proprietà si estende immediatamente agli sketch k-MinHash. Siano

$$\mathcal{S}_k(A) = (a_1, \ldots, a_k), \qquad \mathcal{S}_k(B) = (b_1, \ldots, b_k),$$

due sketch costruiti utilizzando le medesime permutazioni $\pi_1, \ldots, \pi_k$. Lo sketch dell'unione può essere ottenuto mediante il minimo componente per componente:

$$\mathcal{S}_k(A \cup B) = \mathcal{S}_k(A) \oplus \mathcal{S}_k(B), \tag{4}$$

dove

$$\mathcal{S}_k(A) \oplus \mathcal{S}_k(B) = (\min(a_1, b_1), \ldots, \min(a_k, b_k)) .$$

Pertanto, lo sketch k-MinHash di un'unione di insiemi può essere costruito combinando direttamente gli sketch dei singoli insiemi, con un costo $O(k)$. Questa proprietà risulterà particolarmente importante nell'estensione del Neighborhood Search ai grafi temporali, dove sarà necessario recuperare gli sketch dei vicinati relativi a intervalli di tempo differenti senza ricostruire ogni volta gli insiemi originali.

2.3 Neighborhood search mediante LSH

La soluzione si articola in due fasi. Nella fase di *indicizzazione*, eseguita una sola volta, si calcola la firma di ogni nodo a partire dalla sua lista di adiacenza e si inseriscono le b bande in altrettante tabelle di hash. Il costo è $O(k\,m)$, poiché ogni arco contribuisce al calcolo delle firme dei suoi due estremi, e lo spazio occupato è $O(n\,b)$, indipendente dai gradi dei nodi. Nella fase di *interrogazione*, data la query S, si ottiene la sua firma, si esaminano i b bucket corrispondenti e si raccolgono i nodi in essi contenuti come candidati. Ciascun candidato viene quindi verificato, calcolando $\sigma(S, N(v))$, e si restituisce un nodo con similarità almeno τ. Il costo di una query dipende dal numero di candidati e non più da n. Lo schema offre una garanzia probabilistica esplicita: un nodo v con $\sigma(S, N(v)) \geq \tau$ è restituito con probabilità almeno $P(\tau)$, fissabile in fase di progetto tramite b e r. Inoltre, la compressione dei vicinati in firme di dimensione costante neutralizza di fatto l'effetto degli hub, che nella soluzione esatta dominano il costo computazionale.
Adottando la soluzione basata su LSH, è possibile osservare quanto segue:

Osservazione 1. *La procedura di ricerca rimane invariata sia quando la query è un sottoinsieme arbitrario $S \subseteq V$, sia quando essa coincide con il vicinato $N(q)$ di un vertice $q \in V$.*

In entrambi i casi, infatti, la ricerca viene effettuata confrontando la firma MinHash dell'insieme di query con quelle dei vicinati dei vertici del grafo, precedentemente indicizzate mediante LSH. Pertanto, senza perdita di generalità, nel seguito di questa tesi considereremo esclusivamente query costituite da un singolo vertice $q \in V$.

3 Neighborhood Search su Grafi Temporali

L'intera trattazione precedente assume che il grafo non vari nel tempo: il vicinato di un nodo è un insieme ben definito, la similarità tra due nodi è un valore fissato e l'indice costruito resta valido. Molte reti di interesse sono invece dinamiche, poiché le relazioni si formano e si dissolvono. Quando a ciascun arco è associato un istante temporale, o un insieme di istanti, si parla di *grafo temporale*, tipicamente rappresentato come un insieme di triple (u, v, t) che indicano un'interazione tra u e v al tempo t [9, 10]. Nei grafi temporali il vicinato di un nodo dipende dal tempo, e a differenza del caso statico non esiste una definizione canonica di neighborhood search: la dimensione temporale introduce più scelte di modellazione, ciascuna legittima e con conseguenze diverse sul problema risultante. Tali scelte hanno ricadute algoritmiche dirette. Ad esempio il MinHash opera su insiemi fissi, e non è immediato che la costruzione basata su bande si adatti a vicinati che variano nel tempo, né che un indice costruito una volta resti riutilizzabile al variare dell'intervallo di interrogazione.

3.1 Definizioni preliminari

Definiamo ora alcune nozioni preliminari necessarie per comprendere le due varianti del problema di Neighborhood Search su grafi temporali affrontate in questa tesi. In particolare, ci concentreremo sulle definizioni di grafo temporale, vicinato temporale ed istanti temporali [11, 12] , che saranno concetti chiave per lo sviluppo degli algoritmi presentati nei capitoli successivi.

Definizione 3.1.1 (Grafo temporale)**.** *Sia $T \geq 0$ un intero. Un tripla $\mathcal{G} = (V, E, \lambda)$ è detta grafo temporale se $G = (V, E)$ è un grafo, e $\lambda : E \to 2^{[T]}$ è una funzione che associa ad ogni arco $e \in E$ un insieme $\lambda(e)$ di etichette temporali, dette anche timestamps. Indichiamo inoltre con $\mathcal{E}$ l'insieme di archi temporali, definito come*

$$\mathcal{E} = \{(u, v, t) \mid (u, v) \in E \wedge t \in \lambda((u, v))\}.$$

Definizione 3.1.2 (Istanti temporali di un nodo)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ un grafo temporale e sia $v \in V$. Definiamo l'insieme degli istanti temporali in cui v è attivo come*

$$\Lambda(v) = \bigcup_{(v,u)\in E} \lambda((v, u)).$$

In altre parole, $\Lambda(v)$ contiene tutti i timestamp associati agli archi incidenti al nodo v. Inoltre, imponiamo la seguente restrizione: su ogni arco esiste un numero polinomiale in n di etichette.

Osservazione 2. *Dalla definizione di $\Lambda(v)$ possiamo notare che valgolo le seguenti proprietà:*

- $|\Lambda(v)| = |\bigcup_{(v,u)} \lambda(v, u)| \leq \sum_{(v,u)} |\lambda(v, u)|$

- $\sum_{v \in V} |\Lambda(v)| \leq 2|\mathcal{E}|$

Definizione 3.1.3 (Vicinato temporale)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ un grafo temporale e sia $t \in [T]$. Il vicinato temporale di un nodo $v \in V$ al tempo t è l'insieme*

$$N^t(v) = \{u \in V \mid (v, u, t) \in \mathcal{E}\}.$$

In altri termini, $u \in N^t(v)$ se e solo se l'arco temporale (v, u, t) è presente nel grafo.

Definizione 3.1.4 (Vicinato temporale su un intervallo)**.** *Sia $I = [\alpha, \beta] \subseteq [T]$ un intervallo di tempo. Il vicinato temporale di un nodo $v \in V$ su I è*

$$N^I(v) = \bigcup_{t \in I} N^t(v).$$

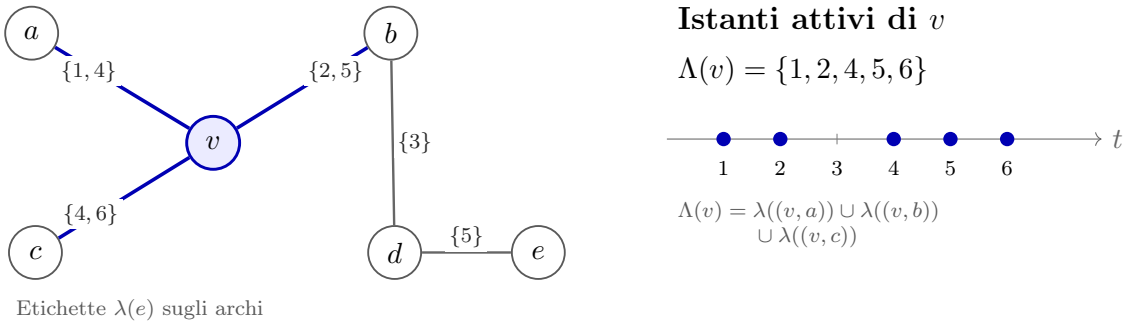


Figura 2: Esempio di grafo temporale $\mathcal{G} = (V, E, \lambda)$.

3.2 Varianti del problema

Passiamo ora alla definizione formale delle due varianti del problema di Neighborhood Search su grafi temporali affrontate in questa tesi. Seppur connesse fra loro, le due varianti del problema presentano differenze sostanziali, portando a complessità diverse. In particolare, la seconda variante richiede di considerare tutti gli intervalli temporali possibili per ogni possibile ampiezza, il che rende la ricerca dell'intervallo corretto molto più complessa e costosa in termini di tempo di esecuzione. Oltre a ciò, per la seconda variante del problema verrà proposta una soluzione euristica, che non risolverà il problema per intero, ma che permetterà di ottenere una soluzione approssimata in maniera efficiente.

Definizione 3.2.1 (NS con intervalli ad ampiezza fissa)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ un grafo temporale, $\tau \in [0, 1]$ una soglia, $\sigma : 2^V \times 2^V \to [0, 1]$ una funzione di similarità e*

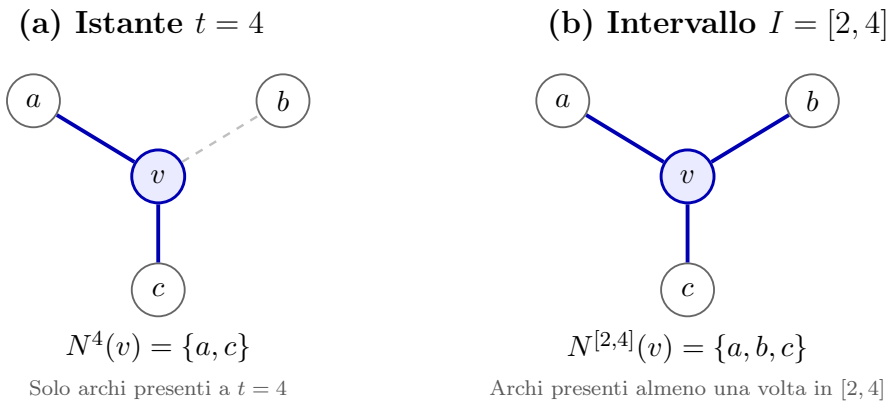


Figura 3: Confronto tra il vicinato temporale istantaneo $N^4(v)$ e quello sull'intervallo $I = [2, 4]$, ottenuto come unione dei vicinati istantanei.

$\mu \geq 0$ un intero. Si vuole processare $\mathcal{G}$ in modo tale da poter rispondere alla seguente query: dato un vertice $v \in V$, ritornare un vertice $u \in V \setminus \{v\}$ tale che esiste un intervallo $I = [t, t + \mu]$, per qualche $t \in [T]$, tale che

$$\sigma\left(N^I(v), N^I(u)\right) \geq \tau.$$

Definizione 3.2.2 (NS con intervalli ad ampiezza variabile)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ un grafo temporale, una soglia $\tau \in [0, 1]$ e $\sigma : 2^V \times 2^V \to [0, 1]$ una funzione di similarità. Si vuole processare $\mathcal{G}$ in modo tale da poter rispondere alla seguente query: dato un vertice $v \in V$ e un intero $\mu \geq 0$, ritornare un vertice $u \in V \setminus \{v\}$ tale che esiste un intervallo $I = [t, t + \mu]$, per qualche $t \in [T]$, tale che*

$$\sigma\left(N^I(v), N^I(u)\right) \geq \tau.$$

3.3 Il ruolo degli sketch

Nel caso statico, uno sketch come MinHash permette di rappresentare il vicinato di un nodo con una firma compatta, dalla quale è possibile stimare la similarità di Jaccard senza confrontare ogni volta gli insiemi completi. Questa rappresentazione è particolarmente utile quando i vicinati sono grandi: lo sketch ha dimensione fissata e può essere utilizzato direttamente nella fase di ricerca basata su LSH. Nei grafi temporali, però, il vicinato da rappresentare dipende anche dall'intervallo richiesto. Sia $\mathcal{G} = (V, E, \lambda)$ un grafo temporale, $v \in V$ un nodo, $I = [a, b] \subseteq T$ una finestra temporale e $N^I(v)$ il vicinato di v attivo in I. Indichiamo con $n_I(v) = |N^I(v)|$ la sua cardinalità. La query che ci interessa è

$$\text{query}(v, I) = \mathcal{S}\big(N^I(v)\big),$$

dove $\mathcal{S}$ è il k-MinHash sketch scelto per rappresentare il vicinato. La soluzione immediata consiste nel ricostruire $N^I(v)$ e calcolarne lo sketch a ogni richiesta. Anche se

la firma finale è compatta, questa procedura deve esaminare gli elementi del vicinato e richiede quindi almeno $\Omega\big(n_I(v)\big)$ tempo per query. Quando cambia I la situazione peggiora ulteriormente: ricalcolare da zero la firma significa scandire nuovamente il vicinato, senza sfruttare il lavoro svolto per intervalli già considerati. Per molte query su finestre diverse, questa strategia può dunque annullare il vantaggio offerto dalla rappresentazione compatta. Il punto chiave è che MinHash è componibile rispetto all'unione: lo sketch dell'unione di insiemi si ottiene combinando i rispettivi sketch. La compibilità del MinHash è stata ampiamente discussa precedentemente, nella Sezione 2.2.1. Questa proprietà permette di puntare a una soluzione più efficiente: invece di ricostruire il vicinato temporale per ogni intervallo, si possono riutilizzare sketch calcolati su porzioni del dominio temporale e combinarli per ottenere la firma richiesta. Non è quindi necessario esaminare nuovamente tutti gli elementi di $N^I(v)$ a ogni query. Una struttura dedicata, e appositamente costruita, che chiameremo *oracolo* $\mathcal{O}$ e che formalizzeremo più avanti (Sezione 4.5), consente di mantenere spazio lineare nel numero di istanti temporali rilevanti e, assumendo dimensione costante per lo sketch, di rispondere alla query in tempo $O\big(\log|\Lambda(v)|\big)$ Il passaggio è dunque da un costo almeno $\Omega(n_I(v))$ per ricostruire e comprimere il vicinato a un costo logaritmico rispetto agli istanti temporali del nodo. La costruzione dell'oracolo e le ipotesi necessarie a questi limiti saranno precisate più avanti.

4 Variante 1: NS con intervalli ad ampiezza fissa

Nella prima variante del problema del Neighborhood Search, gli unici valori conosciuti a priori risultano essere la soglia τ e l'ampiezza dell'intervallo $\mu \geq 0$. Grazie alla conoscenza a priori del fattore μ, possiamo sin da subito conoscere il quantitativo massimo di intervalli possili, che risultano essere $T - \mu$. La soluzione proposta adotta lo stesso principio della soluzione per grafi statici, ovvero utilizzare lo schema filter&verify con il framework LSH. Questo ci permetterà di ridurre, in maniera considerevole, il numero di confronti da effettuare. L' algoritmo risolutivo è suddiviso in due parti: la prima si occupa di effettuare un preprocessing dei dati, in modo da poter creare la struttura LSH; la seconda parte si occupa di rispondere alla query, sfruttando il filtro ottenuto con LSH.

4.1 Algoritmo Brute Force

Prima di passare alla spiegazione dell'algoritmo proposto, è necessario introdurre un algoritmo di riferimento, che risolve il problema in questione in maniera corretta, ma non efficiente. L'algoritmo in questione, che chiameremo algoritmo *Brute Force*, è un algoritmo di ricerca esaustiva, che non sfrutta alcuna struttura dati per ridurre il numero di candidati da confrontare con il nodo target v. L'algoritmo Brute Force opera nel modo seguente:

- Non viene effettuata nessuna operazione di preprocessing dei dati, in quanto non viene costruita alcuna struttura LSH.
- Preso il nodo $v \in V$ passato a query time, per ogni finestra temporale $I_t = [t, t+\mu]$ con $t \in \{1, \ldots, T-\mu\}$ viene recuperato lo sketch $S(v,t)$ e viene calcolata la similarità con tutti gli altri nodi $u \in V \setminus \{v\}$.

Nonostante la semplicità dell'algoritmo, e la sua naturale correttezza, esso risulta essere estremamente inefficiente. La complessità computazionale di tale algoritmo infatti risulta essere:

$$O\Big((T-\mu) \cdot \sum_{u \in V \setminus \{v\}} k \log(|\Lambda(u)|)\Big) \approx O(nTk \log(T))$$

dove n è il numero di nodi, T è la durata della timeline e k è la dimensione dello sketch. Essendo T e n potenzialmente molto grandi, l'algoritmo Brute Force risulta essere poco pratico, e non utilizzabile in scenari reali. Per tale motivo, nel lavoro di tesi viene proposta una soluzione più efficiente, che sfrutta la struttura LSH per ridurre il numero di candidati da confrontare con il nodo target v. Come vedremo nella sezione di esperimenti, l'algoritmo proposto risulterà essere estremamente più efficiente dell'algoritmo Brute Force, pur mantenendo la correttezza della soluzione.

4.2 Algoritmo di generazione dell'indice LSH

L'algoritmo di generazione dell'indice LSH opera in due fasi distinte

Fase 1 - Pruning temporale Per ogni nodo v, recuperiamo l'insieme $\Lambda(v)$. Definiamo un'ulteriore insieme, ovvero l'insieme degli istanti iniziali delle finestre temporalmente rilevanti per v, definito come

$$W(v) = \{t \in \{1, \dots, T - \mu\} \mid [t, t + \mu] \cap \Lambda(v) \neq \emptyset\}.$$

Perchè abbiamo aggiunto questo insieme, perchè grazie a lui possiamo effettuare un **pruning** degli intervalli temporali utili per il nodo v, così da evitare di controllare intervalli non utili. Vale infatti che, se:

$$t \notin W(v) \implies [t, t + \mu] \cap \Lambda(v) = \emptyset$$

e, per definizione di $\Lambda(v)$,

$$N^{I_t}(v) = \emptyset.$$

Pertanto tali finestre possono essere escluse dalla ricerca. La query può quindi essere limitata ai soli istanti temporali $t \in W(v)$, invece di considerare tutti gli istanti $t \in \{1, \dots, T - \mu\}$. L'analisi di questo insieme $W(v)$ e della sua cardinalità è fondamentale per comprendere l'effettivo beneficio del pruning temporale. In particolare, la dimensione di $W(v)$ dipende dalla distribuzione temporale degli istanti appartenenti a $\Lambda(v)$ e può variare significativamente in funzione della densità temporale degli eventi associati al nodo v.

Fase 2 - Indicizzazione delle coppie nodo–tempo Completato il pruning temporale, costruiamo l'indice LSH sulle sole coppie nodo–tempo rilevanti. Sia $\mathcal{O}$ l'oracolo che permette di recuperare gli sketch dei vicinati temporali (preannunciato in 3.3 e successivamente descritto nella sezione 4.5) . A ogni coppia (v, t), con $t \in W(v)$, recuperiamo il vicinato del nodo nella finestra $I_t = [t, t + \mu]$ e il relativo sketch tramite l'oracolo $\mathcal{O}$:

$$S(v, t) = \text{Sketch}\left(N^{I_t}(v)\right).$$

Gli sketch, di dimensione k, vengono suddivisi in b band di r righe ciascuna, tale per cui vale la relazione $k = br$. Indichiamo ora con $B_j(v, t)$ la j-esima band dello sketch $S(v, t)$. Tutte le coppie vengono memorizzate in un unico indice, composto da b tabelle hash:

$$\mathcal{L} = \{H_1, \dots, H_b\}.$$

Nella tabella H_j, la chiave non sarà quella standard di LSH, ma sarà composta dall'hash della band $B_j(v, t)$ e dall'istante iniziale della finestra, nel modo seguente:

$$K_j(v, t) = (Hash_j(B_j(v, t)), t).$$

Ovviamente, il bucket corrispondente contiene direttamente il nodo v:

$$H_j[K_j(v,t)] \to v.$$

Includere t nella chiave fa sì che ogni bucket raccolga soltanto nodi associati alla stessa finestra temporale; le collisioni LSH selezionano così candidati con vicinati potenzialmente simili nello stesso intervallo.
L'Algoritmo 1 riporta lo pseudocodice dettagliato della costruzione dell'indice.

Algorithm 1 Costruzione dell'indice LSH

Require: Grafo temporale $\mathcal{G} = (V, E, \lambda)$, ampiezza μ, oracolo $\mathcal{O}_v$
Ensure: Tabelle hash $H_1, \ldots, H_b$

```
function BuildLSH-Index(G, μ)
    for all j = 1, ..., b do
        H_j ← ∅
    end for
    for all v ∈ V do
        Calcola e memorizza W(v)
        for all t ∈ W(v) do
            S(v,t) ← O_v.query([t, t+μ])
            for all j = 1, ..., b do
                h ← Hash_j(B_j(v,t))
                key ← (h,t)
                H_j[key].insert(v)
            end for
        end for
    end for
    return H_1, ..., H_b
end function
```

Il ciclo su $W(v)$ applica il pruning temporale: si indicizzano solo le finestre che contengono almeno un istante di $\Lambda(v)$. Per ogni finestra, lo sketch viene diviso in band e ciascuna band determina un bucket in una delle tabelle. La chiave include t, così la ricerca dei candidati confronta nodi solo nella stessa finestra; le collisioni sulle band servono invece a selezionare nodi con sketch potenzialmente simili.

4.3 Algoritmo di risposta alla query

Data una query sul nodo v, esaminiamo gli istanti $t \in W(v)$. Per ciascuno recuperiamo lo sketch $S(v,t)$ dalla struttura temporale, con costo $O(k \log |\Lambda(v)|)$, e usiamo le sue b band per interrogare le tabelle LSH. Unendo i risultati dei lookup otteniamo l'insieme

dei candidati per la finestra I_t:

$$C(v,t) = \bigcup_{j=1}^{b} H_j[K_j(v,t)].$$

Escludiamo v stesso e confrontiamo lo sketch di ciascun candidato $u \in C(v,t)$ con $S(v,t)$. Se

$$\sigma\left(S(v,t), S(u,t)\right) \geq \tau,$$

la query restituisce la coppia (u,t) e termina con esito positivo. Se nessun candidato supera la soglia in alcuna finestra esaminata, l'esito è negativo. L'algoritmo 2 riporta lo pseudocodice dettagliato della ricerca dei candidati e della verifica della soglia per ciascuna finestra considerata.

Algorithm 2 Risposta alla query

Require: Nodo $v \in V$, ampiezza μ, insieme $W(v)$, oracolo $\mathcal{O}$, tabelle $H_1, \ldots, H_b$
Ensure: Una coppia (u,t) valida, oppure **False**

```
1:  function QUERY(v)
2:      for all t ∈ W(v) do
3:          S(v,t) ← O_v.query([t, t+μ])
4:          C(v,t) ← ∅
5:          for all j = 1, ..., b do
6:              h ← Hash_j(B_j(v,t))
7:              key ← (h,t)
8:              C(v,t) ← C(v,t) ∪ H_j[key]
9:          end for
10:         C(v,t) ← C(v,t) \ {v}
11:         for all u ∈ C(v,t) do
12:             S(u,t) ← RT_u.query([t, t+μ])
13:             if σ(S(v,t), S(u,t)) ≥ τ then
14:                 return (u,t)
15:             end if
16:         end for
17:     end for
18:     return False
19: end function
```

4.4 Analisi della complessità

Per poter effettuare un'analisi sensata e concreta della complessità dei due algoritmi proposti, come prima cosa è necessario analizzare la dimensione dell'insieme $W(v)$,

in quanto esso rappresenta il numero di finestre temporali che l'algoritmo deve considerare per ogni nodo v. Dopo aver analizzato la dimensione di $W(v)$, l'analisi della complessità verrà effettuata separando l'algoritmo per l'indice LSH con l'algoritmo di risposta alla query. A tal proposito, possiamo fornire il seguente teorema, che preannuncia i bound finali ottenuti.

Teorema 4.1 (Bound di complessità)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ il grafo temporale in input, $\tau \in [0, 1]$ il valore soglia e $\mu \geq 0$ l'ampiezza della finestra temporale. L'algoritmo proposto*

- *preprocessa $\mathcal{G}$ in tempo $O(\sum_{v \in V} |W(v)|(k \cdot \log |\Lambda(v))| + b)$*
- *utilizza spazio ausiliare pari a $O(b \cdot \sum_{v \in V} |W(v)|)$*
- *risponde ad una query in tempo $O(|W(v)|(k \cdot \log |\Lambda(v)| + b) + C_v \cdot k)$, con $C_v = \sum_{t \in W(v)} |C(v, t)|$*

Analisi della dimensione di $W(v)$ Per analizzare il costo effettivo del pruning temporale introdotto dall'insieme $W(v)$, è necessario determinarne la cardinalità in funzione della struttura temporale dell'insieme $\Lambda(v)$. Sia

$$\Lambda(v) = \{\lambda_1, \lambda_2, \ldots, \lambda_m\}, \qquad \lambda_1 < \lambda_2 < \cdots < \lambda_m,$$

dove $m = |\Lambda(v)|$. Consideriamo un generico istante $\lambda_i \in \Lambda(v)$. Tale istante appartiene alla finestra $[t, t + \mu]$ se e solo se $t \leq \lambda_i \leq t + \mu$. Equivalentemente, $\lambda_i - \mu \leq t \leq \lambda_i$. Pertanto, a ogni elemento λ_i di $\Lambda(v)$ possiamo associare l'intervallo

$$J_i = [\lambda_i - \mu, \lambda_i] \cap \{1, \ldots, T - \mu\},$$

che rappresenta l'insieme degli istanti iniziali t per i quali la finestra $[t, t + \mu]$ contiene l'istante λ_i. Ne segue che

$$W(v) = \bigcup_{i=1}^{m} J_i.$$

Questa rappresentazione permette di determinare un bound superiore sulla cardinalità di $W(v)$. Ogni intervallo J_i contiene al più $\mu + 1$ istanti interi. Pertanto, utilizzando il **principio dello Subadditività della Misura** otteniamo:

$$|W(v)| = \left| \bigcup_{i=1}^{m} J_i \right| \leq \sum_{i=1}^{m} |J_i| \leq m(\mu + 1) = (\mu + 1)|\Lambda(v)|.$$

D'altra parte, per costruzione,

$$W(v) \subseteq \{1, \ldots, T - \mu\} \implies |W(v)| \leq T - \mu.$$

Combinando i due bound si ottiene

$$|W(v)| \leq \min\{T - \mu, (\mu + 1)|\Lambda(v)|\}.$$

Il bound precedente è stretto rispetto al contributo di $\Lambda(v)$. Infatti, consideriamo il caso in cui gli elementi di $\Lambda(v)$ siano sufficientemente distanti, ovvero

$$\lambda_{i+1} - \lambda_i > \mu \qquad \forall i = 1, \ldots, m-1.$$

In questo caso gli intervalli J_i sono disgiunti e, ignorando gli eventuali effetti dovuti ai bordi della timeline, ciascuno contiene esattamente $\mu + 1$ istanti. Si ottiene quindi $|W(v)| = (\mu + 1)|\Lambda(v)|$. Al contrario, se gli elementi di $\Lambda(v)$ sono temporalmente ravvicinati, gli intervalli J_i presentano una forte sovrapposizione. Per caratterizzare tale fenomeno, definiamo il gap temporale tra due elementi consecutivi di $\Lambda(v)$ come

$$d_i = \lambda_{i+1} - \lambda_i.$$

Quando $d_i \leq \mu$, gli intervalli J_i e J_{i+1} si sovrappongono; quando invece $d_i > \mu$, essi risultano disgiunti. In particolare, nel caso di massima densità temporale, ossia quando

$$d_i = 1 \qquad \forall i,$$

si ha

$$|W(v)| = (\mu + 1) + (m - 1) = m + \mu \implies |W(v)| = |\Lambda(v)| + \mu.$$

Pertanto, a seconda della distribuzione temporale degli elementi di $\Lambda(v)$, si possono distinguere due configurazioni:

$$|\Lambda(v)| + \mu \;\leq\; |W(v)| \;\leq\; \min\{T - \mu, (\mu + 1)|\Lambda(v)|\},$$

In particolare, se μ è considerato un parametro fissato a priori, allora $(\mu + 1) = O(1)$ e il bound può essere riscritto come

$$|W(v)| = O\left(\min\{T, |\Lambda(v)|\}\right).$$

Tale risultato mostra che il pruning temporale può ridurre significativamente il numero di finestre considerate quando $|\Lambda(v)| \ll T$, mentre nel caso peggiore, quando $\Lambda(v)$ è sufficientemente densa, si può avere

$$|W(v)| = \Theta(T).$$

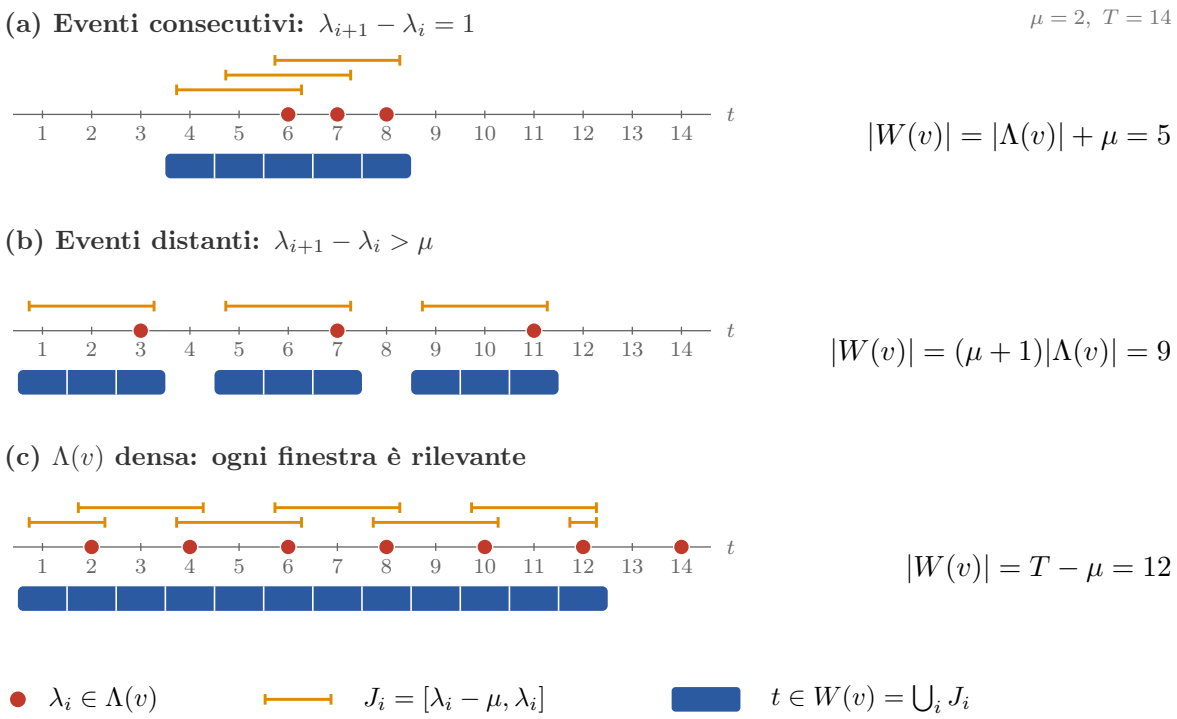


Figura 4: Analisi della dimensione di $W(v)$,utilizzando valori di test $T = 14, \mu = 2, T - \mu = 12$

La Figura 4 mostra come la cardinalità di $W(v)$ dipenda dalla distribuzione temporale degli elementi di $\Lambda(v)$. Si distinguono tre casi: (a) eventi consecutivi, per cui gli intervalli J_i si sovrappongono e $|W(v)| = |\Lambda(v)| + \mu$; (b) eventi sufficientemente distanziati, per cui gli intervalli sono disgiunti e $|W(v)| = (\mu + 1)|\Lambda(v)|$; (c) eventi distribuiti in modo da coprire tutte le finestre ammissibili, per cui $|W(v)| = T - \mu$. Pertanto, l'efficacia del pruning temporale dipende dalla distribuzione degli eventi lungo la timeline. Dopo aver analizzato la dimensione di $W(v)$, possiamo passare all'analisi di complessità per l'indice LSH e per la query.

Complessità della costruzione dell'indice LSH. Sia $n = |V|$ il numero di nodi e, per ogni nodo v, siano $m_v = |\Lambda(v)|$ e $w_v = |W(v)|$. La costruzione dell'indice LSH considera esclusivamente le coppie nodo–tempo (v, t), con $v \in V$ e $t \in W(v)$. Il numero complessivo di coppie effettivamente indicizzate è pertanto

$$M_W = \sum_{v \in V} |W(v)|.$$

Per ogni coppia (v, t), l'algoritmo recupera la signature $S(v, t) = \text{Sketch}(N^{I_t}(v))$ mediante la struttura temporale sottostante. Il costo del recupero della signature è $O(k \log m_v)$. Successivamente la signature viene suddivisa in b band e, per ciascuna

band, viene calcolata la funzione hash e viene effettuato l'inserimento nella corrispondente hash table. Assumendo costo atteso $O(1)$ per le operazioni di hashing e inserimento, questa fase richiede tempo $O(b)$. Il costo associato alla singola coppia (v, t) è quindi

$$O(k \log m_v + b).$$

Considerando tutte le coppie appartenenti a $W(v)$, la complessità temporale relativa alla **costruzione dell'indice** (*building time*) risulta essere pari a

$$O\left(\sum_{v \in V} |W(v)| \, (k \log m_v + b)\right).$$

Utilizzando il bound precedentemente ottenuto sulla cardinalità di $W(v)$, si ottiene

$$O\left(\sum_{v \in V} \min\{T - \mu, (\mu + 1)m_v\} \, (k \log m_v + b)\right).$$

Poiché μ è considerato fissato a priori, nel regime in cui $(\mu+1)m_v \ll T-\mu$, il termine $|W(v)|$ può essere limitato da $O(m_v)$ e la complessità diventa

$$O\left(\sum_{v \in V} m_v \, (k \log m_v + b)\right).$$

Definendo $m = \max_{v \in V} |\Lambda(v)|$, si può inoltre ottenere il seguente bound semplificato:

$$O\left(n \min\{T - \mu, (\mu + 1)m\}(k \log m + b)\right).$$

Nel caso peggiore, $W(v)$ può coincidere con l'intero insieme $\{1, \ldots, T - \mu\}$ per ogni nodo. In tale situazione si ha $|W(v)| = T - \mu$ e la complessità torna ad essere

$$O\left(n(T - \mu)(k \log m + b)\right).$$

Pertanto il pruning basato su $W(v)$ non modifica il worst-case asintotico della costruzione dell'indice, ma può ridurre il costo effettivo quando gli insiemi $\Lambda(v)$ sono temporalmente poco densi. Per quanto riguarda lo spazio occupato dall'indice, ogni coppia (v, t) viene inserita in tutte le b hash table. Lo **spazio** richiesto è quindi

$$O\left(b \sum_{v \in V} |W(v)|\right) = O\left(b \sum_{v \in V} \min\{T - \mu, (\mu + 1)m_v\}\right).$$

Complessità della query. Consideriamo ora una query sul nodo v. Grazie al pruning temporale, l'algoritmo non deve esaminare tutte le $T - \mu$ finestre temporali, ma soltanto gli istanti appartenenti a $W(v)$. Poniamo nuovamente $w_v = |W(v)|$.

Per ogni $t \in W(v)$, la signature $S(v,t)$ viene recuperata dalla struttura temporale sottostante. Il costo complessivo del recupero delle signature è pertanto

$$O\left(w_v k \log m_v\right).$$

Per ogni finestra vengono inoltre effettuati b lookup nelle hash table LSH. Assumendo costo atteso $O(1)$ per ciascun lookup, il costo complessivo di questa fase è $O(w_v b)$. Il costo associato alla generazione dei candidati è quindi

$$O\left(w_v(k \log m_v + b)\right).$$

Sia ora $C(v,t)$ l'insieme dei candidati prodotti dall'indice LSH nella finestra t e sia $C_{\text{sim}} = O(k)$ il costo della verifica esatta della similarità tra due vicinati. La verifica di tutti i candidati richiede tempo

$$O\left(\sum_{t \in W(v)} |C(v,t)| k\right).$$

La complessità complessiva della query è pertanto

$$O\left(|W(v)|(k \log |\Lambda(v)| + b) + \sum_{t \in W(v)} |C(v,t)| k\right).$$

Utilizzando il bound sulla cardinalità di $W(v)$ si ottiene

$$O\left(\min\{T - \mu, (\mu+1)|\Lambda(v)|\}(k \log |\Lambda(v)| + b) + \sum_{t \in W(v)} |C(v,t)| k\right).$$

Se μ è fissato e ci troviamo nel regime in cui $(\mu+1)|\Lambda(v)| \ll T - \mu$, allora $|W(v)| = O(|\Lambda(v)|)$ e il costo della componente di recupero delle signature e di accesso all'indice diventa

$$O\left(|\Lambda(v)|(k \log |\Lambda(v)| + b)\right).$$

Definendo $C_v = \sum_{t \in W(v)} |C(v,t)|$, la complessità temporale di risposta alla query può essere infine espressa in forma compatta come

$$O\left(|\Lambda(v)|(k \log |\Lambda(v)| + b) + C_v k\right).$$

Confronto tra $|\Lambda(v)|$ e $|W(v)|$ ed effetto del pruning. Il confronto tra la cardinalità di $\Lambda(v)$ e quella di $W(v)$ permette di quantificare l'effettivo beneficio del pruning temporale. Ogni elemento $\lambda \in \Lambda(v)$ può rendere rilevanti fino a $\mu + 1$ diversi

istanti iniziali di finestra. Se gli elementi di $\Lambda(v)$ sono temporalmente molto ravvicinati, gli intervalli associati $[\lambda - \mu, \lambda]$ si sovrappongono fortemente. Nel caso limite di eventi consecutivi, si ottiene $|W(v)| = |\Lambda(v)| + \mu$. Per μ fissato, questo implica

$$|W(v)| = \Theta(|\Lambda(v)|).$$

In tale situazione il numero di finestre esaminate cresce linearmente con il numero di istanti temporalmente rilevanti del nodo, anziché con la durata complessiva T della timeline. Se invece gli elementi di $\Lambda(v)$ sono separati temporalmente da più di μ, gli intervalli associati non si sovrappongono e, salvo effetti di bordo, si ha $|W(v)| = (\mu + 1)|\Lambda(v)|$. Infine, nel caso in cui $\Lambda(v)$ sia sufficientemente densa da rendere rilevante praticamente ogni possibile finestra, si verifica

$$W(v) = \{1, \ldots, T - \mu\} \implies |W(v)| = T - \mu.$$

In questo caso il pruning non produce alcuna riduzione asintotica del numero di finestre analizzate. Possiamo quindi riassumere l'effetto del pruning attraverso il rapporto

$$\frac{T - \mu}{|W(v)|}.$$

Tale quantità rappresenta, in termini di numero di finestre analizzate, il fattore di riduzione ottenuto rispetto alla scansione completa della timeline. Quando $|W(v)| \ll T - \mu$, il fattore di riduzione è elevato e il pruning risulta efficace. In particolare, se μ è fissato e $|\Lambda(v)| \ll T$, nei casi in cui $|W(v)| = O(|\Lambda(v)|)$, il numero di finestre esaminate passa da

$$O(T) \to O(|\Lambda(v)|).$$

Pertanto, il vantaggio del metodo non è garantito nel worst case, ma dipende dalla densità temporale degli istanti appartenenti a $\Lambda(v)$. Il caso favorevole è quello in cui il numero di istanti temporalmente rilevanti per il nodo è significativamente inferiore alla durata complessiva della timeline. In tale scenario, l'insieme $W(v)$ costituisce un efficace filtro temporale che riduce il numero di coppie nodo–tempo sulle quali deve essere applicato l'indice LSH.

4.5 Oracolo $\mathcal{O}$ per il recupero degli sketch

In questa sezione analizzeremo la struttura dati proposta in questa tesi. Tale struttura è stata utilizzata in precedenza, nella soluzione proposta nella Sezione 4' per la variante 1 del problema. In particolare, evidenzieremo i seguenti punti:

- mostrare come vengono aggiunte le informazioni satellite al rangre tree
- mostrare come viene risolta la query, con argomentazioni sulla correttezza
- analizzare spazio, building time e complessità della struttura dati sul singolo nodo

L'obiettivo principale è quello di rappresentare in modo compatto e manipolabile i vicinati temporali dei nodi, consentendo di rispondere rapidamente a query del tipo:

$$\text{dato } v \in V \text{ e } I = [\alpha, \beta] \subseteq [T], \text{ restituire } \text{Sketch}(N^I(v)).$$

La struttura proposta combina due ingredienti fondamentali, ovvero:

- un *Range Tree* costruito sui timestamp attivi di ciascun nodo;
- un insieme di *Data Sketch*, cioè strutture capaci di rappresentare un insieme in modo compatto.

In particolare, la struttura è progettata per supportare diversi tipi di sketch, purché mantengano la proprietà di *componibilità*. Tra i possibili sketch, in questa tesi è stato preso in esame il **MinHash**. Il MinHash è particolarmente adatto per il nostro contesto, in quanto permette di combinare gli sketch dei vicinati temporali in modo efficiente.

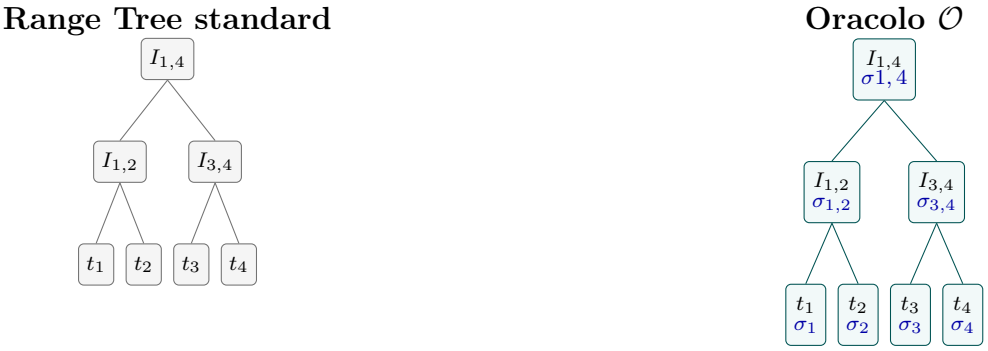


Figura 5: Confronto tra un Range Tree standard, e l'oracolo $\mathcal{O}$.

4.5.1 Range tree

Definiamo formalmente la struttura **Range Tree**, per poi vederne il funziomento relativo alle **Range Query**. Il Range Tree, come abbiamo visto in precedenza, sarà il nostro *building block* principale per la costruzione dell'oracolo. La struttura dati Range Tree fu introdotta da J.L. Bentely [13]

Definizione 4.5.1 (Range Tree)**.** . *Sia $P = \{p_1, \ldots, p_n\}$ un insieme di punti dotati di una coordinata temporale*

$$\tau : P \to [T],$$

dove $\mathbb{T}$ è un insieme totalmente ordinato. Supponiamo, senza perdita di generalità, che

$$\tau(p_1) \leq \tau(p_2) \leq \cdots \leq \tau(p_n).$$

Un range tree *unidimensionale su P è un albero binario di ricerca bilanciato $T = (V, E)$ avente le seguenti proprietà.*

- *Ad ogni nodo $v \in V$ è associato un sottoinsieme $P_v \subseteq P$.*
- *La radice r rappresenta l'intero insieme: $P_r = P$.*
- *Se v è un nodo interno con figli v_L e v_R, allora $P_v = P_{v_L} \dot\cup P_{v_R}$ dove $\dot\cup$ denota unione disgiunta.*

Inoltre, vale la proprietà di ordinamento:

$$\forall p \in P_{v_L},\ \forall q \in P_{v_R}, \qquad \tau(p) \leq \tau(q).$$

Ad ogni nodo v associamo l'intervallo temporale

$$I_v = [\ell_v, r_v],$$

dove

$$\ell_v = \min_{p \in P_v} \tau(p), \qquad r_v = \max_{p \in P_v} \tau(p).$$

Pertanto,

$$P_v = \{p \in P \mid \tau(p) \in I_v\}.$$

Il sottoalbero radicato in v rappresenta quindi un intervallo contiguo dell'ordinamento temporale globale. Poiché l'albero è bilanciato, la sua altezza soddisfa

$$h(T) = O(\log n).$$

4.5.1.1 Range Query

Data una finestra temporale $Q = [a, b] \subseteq [T]$, la corrispondente range query è definita come

$$\mathrm{RQ}(P, Q) = \{p \in P \mid a \leq \tau(p) \leq b\}.$$

L'obiettivo del range tree è rappresentare la risposta alla query mediante un insieme di sottoalberi disgiunti, detti *canonical nodes*. Definiamo l'insieme dei canonical nodes della query Q come

$$C_T(Q) \subseteq V$$

tale che

$$\mathrm{RQ}(P, Q) = \dot{\bigcup_{v \in C_T(Q)}} P_v.$$

I sottoinsiemi P_v associati ai canonical nodes sono quindi disgiunti e la loro unione coincide esattamente con l'insieme degli elementi che soddisfano la query. Per un range tree bilanciato unidimensionale vale

$$|C_T(Q)| = O(\log n).$$

La ricerca della decomposizione canonica viene effettuata individuando i cammini dalla radice verso gli estremi a e b. Tale formulazione di *decomposizione canonica* è stata introdotta qui [14]. Durante la discesa, ogni sottoalbero interamente contenuto nell'intervallo $[a, b]$ viene aggiunto alla decomposizione senza essere ulteriormente visitato. Di conseguenza, una query può essere rappresentata come

$$Q \longrightarrow C_T(Q) = \{v_1, \ldots, v_m\}, \qquad m = O(\log n).$$

La proprietà fondamentale è quindi

$$\mathrm{RQ}(P, Q) = \dot{\bigcup_{i=1}^{m}} P_{v_i}.$$

Se gli elementi devono essere esplicitamente restituiti, il costo della query è

$$O(\log n + k),$$

dove k è il numero di elementi restituiti. Se invece la query richiede un'aggregazione e tale aggregazione è già memorizzata nei nodi, non è necessario visitare i singoli elementi.

4.5.2 Definizione della struttura dati

Per ogni nodo $v \in V$, siano $\Lambda(v)$ e $N^t(v)$ i due insiemi definiti rispettivamente nella Definizione 3.1.2 e Definizione 3.1.3
La struttura dati costruisce, per ciascun nodo v, un Range Tree sui timestamp di $\Lambda(v)$.
Più formalmente, il Range Tree associato a v è composto da:

- una foglia per ogni timestamp $t_i \in \Lambda(v)$;
- un nodo interno per ogni intervallo di timestamp ottenuto come unione di foglie contigue.

Ogni foglia i contiene lo sketch del vicinato temporale del nodo v al tempo t_i:

$$\mathrm{leaf}_i = \mathrm{Sketch}(N^{t_i}(v)).$$

Ogni nodo interno rappresenta un intervallo di tempo $I = [\alpha, \beta] \subseteq [T]$ e contiene lo sketch ottenuto dalla combinazione degli sketch dei figli:

$$\mathrm{node}(I) = \bigoplus_{t \in I \cap \Lambda(v)} \mathrm{Sketch}(N^t(v)),$$

dove $\oplus$ denota l'operazione di merge dei Data Sketch. In particolare, il valore memorizzato in un nodo interno è il risultato della combinazione dei vicinati temporali dei

nodi figli, e quindi rappresenta uno sketch del vicinato temporale sul corrispondente intervallo di tempo: $\text{node}(I) = \text{Sketch}(N^I(v))$, dove $N^I(v) = \bigcup_{t \in I} N^t(v)$. La radice del Range Tree rappresenta l'intervallo completo di tempi in cui il nodo è attivo:

$$\text{root}(v) = \text{Sketch}(N^{[t_1, t_{|\Lambda(v)|}]}(v)).$$

La struttura dati progettata è quindi composta da due livelli:

- **Layer temporale**: un Range Tree per ogni nodo v, ordinato sui timestamp di $\Lambda(v)$;
- **Layer di sketch**: un Data Sketch mergeabile associato a ogni nodo dell'albero, che rappresenta la combinazione dei vicinati temporali degli intervalli di tempo corrispondenti.

4.5.3 Query sulla struttura dati

La query supportata dalla struttura dati è la seguente:

$$\text{Query}(v, I) = \text{Sketch}(N^I(v)).$$

Per rispondere a una query su un intervallo di tempo $I = [\alpha, \beta]$, la struttura Range Tree seleziona un insieme di nodi canonici, cioè nodi dell'albero corrispondenti a intervalli disgiunti la cui unione è pari a tutto I. Questi nodi vengono poi combinati tramite l'operazione di merge del Data Sketch:

$$\text{Sketch}(N^I(v)) = \bigoplus_{J \in \mathcal{C}(I)} \text{Sketch}(N^J(v)),$$

dove $\mathcal{C}(I)$ è l'insieme dei nodi canonici che ricoprono I. Poiché un Range Tree standard rappresenta un intervallo di tempo con un numero di nodi canonici di ordine

$$O(\log |\Lambda(v)|),$$

la query richiede di combinare un numero di sketch pari a

$$O(\log |\Lambda(v)|).$$

ottenendo così un tempo di risposta della query proporzionale a

$$O(k \cdot \log |\Lambda(v)|),$$

dove k è il costo computazionale per combinare due sketch.

4.5.3.1 Caso speciale di query: tempo $O(1)$ con 1-MinHash

Nel caso particolare in cui lo sketch utilizzato sia **1-MinHash**, è possibile ottenere una soluzione più efficiente, rispetto alla soluzione precedente, sfruttando una riduzione al

problema della **Range Minimum Query (RMQ)**.
Il problema RMQ consiste, dato un array statico $A[1, \ldots, T]$, nel determinare, per una query costituita da un intervallo $[l, r]$, il minimo valore contenuto nell'intervallo:

$$\mathrm{RMQ}(A, l, r) = \min_{l \leq i \leq r} A[i].$$

Nel seguito si assume disponibile una struttura dati RMQ utilizzata come *black box*, con le seguenti garanzie: a fronte di un preprocessing lineare, essa occupa $O(T)$ spazio e permette di rispondere a ciascuna query in tempo $O(1)$. Le formulazioni di *RMQ* e le rispettive ottimizzazioni/perfezionamenti sono state introdotte rispettivamente in [15, 16]. Consideriamo ora il caso in cui lo sketch associato a ciascun timestamp sia un 1-MinHash. Per un insieme S, il relativo sketch è definito come

$$SK(S) = \min_{x \in S} h(x),$$

dove h è una funzione hash. Una proprietà fondamentale del 1-MinHash è che il merge di due sketch coincide con l'operatore minimo:

$$SK(S_1 \cup S_2) = \min(SK(S_1), SK(S_2)).$$

Pertanto, per ciascun nodo v, è possibile definire un array temporale

$$A_v[t] = SK(N^t(v)), \quad t \in [T].$$

Lo sketch relativo a un intervallo temporale $I = [l, r]$ è ottenuto effettuando il merge degli sketch di tutti gli istanti appartenenti all'intervallo. Grazie alla proprietà del 1-MinHash, tale operazione può essere riscritta come

$$SK(N^{[l,r]}(v)) = \bigoplus_{t=l}^{r} SK(N^t(v)) = \min_{l \leq t \leq r} SK(N^t(v)) = \min_{l \leq t \leq r} A_v[t].$$

L'ultima espressione è esattamente una Range Minimum Query sull'array A_v. Di conseguenza,

$$SK(N^{[l,r]}(v)) = \mathrm{RMQ}(A_v, l, r)$$

e la query dello sketch può essere eseguita direttamente mediante la struttura RMQ, senza dover attraversare il Range Tree e recuperare esplicitamente i relativi nodi canonici. Assumendo quindi la struttura RMQ come *black box*, la query $(v, [l, r])$ richiede $O(1)$ tempo, a fronte di $O(T)$ spazio per la struttura RMQ associata al nodo v. Si ottiene pertanto il seguente miglioramento:

$$O(k \log(|\Lambda(v)|)) \longrightarrow O(1)$$

nel caso del 1-MinHash, per il quale $k = 1$.

Il miglioramento deriva dal fatto che il Range Tree risolve la query mediante una decomposizione dell'intervallo in $O(\log T)$ nodi canonici, mentre la struttura RMQ incorpora tale decomposizione nella fase di preprocessing e consente di recuperare direttamente il minimo dell'intervallo in tempo costante. In altre parole, nel caso del 1-MinHash, il problema di recuperare e fondere gli sketch di un intervallo temporale è esattamente equivalente a una RMQ sulla sequenza degli sketch alle singole unità temporali.

4.5.4 Analisi dei costi

Per poter effettuare un'analisi concreta della struttura dati Oracolo, è necessario analizzare tre concetti fondamentali:

- il tempo di costruzione (*building time*);
- lo spazio occupato;
- il tempo di query.

In questa sezione verranno forniti i dettagli di ciascun concetto, con particolare attenzione alla dipendenza dai parametri del grafo temporale e dalle caratteristiche degli sketch utilizzati.

Tempo di costruzione

Sia $d_v = |\Lambda(v)|$ il numero di timestamp in cui il nodo v è attivo. Per costruire lo sketch di ciascun vicinato temporale istantaneo $N^t(v)$, supponiamo che il costo sia pari a k. Il costo per costruire tutti i nodi foglia di v è quindi

$$O(d_v \cdot k).$$

Successivamente, il Range Tree richiede di combinare gli sketch dei figli per costruire i nodi interni. Se il numero di figli per nodo è costante, la complessità totale di costruzione dell'albero per il nodo v è

$$O(d_v \log d_v \cdot k),$$

dove k è il costo di merge di due sketch. Di conseguenza, il costo totale di costruzione per il nodo v può essere espresso come

$$T_{\text{build}}(v) = O(d_v \log d_v \cdot k) = O(k|\Lambda(v)| \log(|\Lambda(v)|)).$$

Per l'intero grafo, basta sommare i costi per ciascun nodo, ottenendo:

$$T_{\text{build}}(\mathcal{G}) = \sum_{v \in V} O(d_v \log d_v \cdot k).$$

Se si assume che la somma dei timestamp attivi sui nodi sia proporzionale al numero di archi temporali, cioè $\sum_{v \in V} d_v = O(|\mathcal{E}|)$, allora si ottiene

$$T_{\text{build}}(\mathcal{G}) = O(k \cdot |\mathcal{E}| \log |\mathcal{E}|).$$

Spazio occupato

Sia s la dimensione di memoria di uno sketch. Il numero di sketch memorizzati nella struttura per un nodo v è pari al numero di nodi del Range Tree, cioè $O(d_v)$. Quindi lo spazio occupato per il nodo v è

$$S(v) = O(d_v \cdot s).$$

Per l'intero grafo

$$S(\mathcal{G}) = \sum_{v \in V} O(d_v \cdot s) = O\left(s \cdot \sum_{v \in V} d_v\right).$$

Poiché

$$\sum_{v \in V} d_v = O(|\mathcal{E}|),$$

si ottiene

$$S(\mathcal{G}) = O(|\mathcal{E}| \cdot s).$$

Nel caso in cui il Data Sketch abbia dimensione fissata, cioè $s = O(1)$, lo spazio totale diventa lineare nel numero di archi temporali:

$$S(\mathcal{G}) = O(|\mathcal{E}|).$$

Tempo di query

Una query su un intervallo temporale $I = [\alpha, \beta]$ per un nodo v richiede di selezionare i nodi canonici del Range Tree che ricoprono I e di combinare gli sketch corrispondenti. Il numero dei nodi canonici sappiamo essere pari a:

$$O(\log d_v),$$

quindi il costo di query per un singolo nodo è

$$T_{\text{query}}(v) = O(\log d_v \cdot k).$$

Se lo sketch ha dimensione fissa, allora $k = O(1)$ e quindi

$$T_{\text{query}}(v) = O(\log d_v).$$

In termini di notazione più esplicita, la query restituisce

$$\text{Query}(v, I) = \bigoplus_{J \in \mathcal{C}(I)} \text{Sketch}(N^J(v)),$$

ottenendo lo sketch del vicinato temporale di v sull'intervallo I.

5 Variante 2: NS con intervalli ad ampiezza variabile

Analizziamo ora l'ultima versione del problema. Qui l'unico parametro noto a priori è la soglia τ, mentre l'ampiezza $\mu \geq 0$ viene fornita a query time insieme al nodo v. Come nelle sezioni precedenti, assumiamo che la funzione di similarità σ sia l'indice di Jaccard e che le signature siano sketch MinHash di dimensione k, ottenuti con k permutazioni indipendenti $\pi_1, \ldots, \pi_k$ di V:

$$S_v(I) = \big(h_v^1(I), \ldots, h_v^k(I)\big), \qquad h_v^i(I) = \min \pi_i\big(N^I(v)\big).$$

Chiamiamo $h_v^i(I)$ l'i-esimo **blocco** della signature.

5.1 Dal problema originale al sottoproblema

Perché le soluzioni precedenti non si estendono. Nel caso del problema ad ampiezza fissa (Sezione 4) μ è fissato e l'indice è costruito sulle coppie nodo–tempo (v, t), con chiave $K_j(v, t) = (\mathrm{Hash}_j(B_j(v, t)), t)$. Tale chiave dipende da μ, perché la signature $S(v, t)$ è calcolata sulla finestra $[t, t + \mu]$. Nel problema attuale μ è noto solo a query time, quindi un indice dello stesso tipo dovrebbe coprire tutti i valori $\mu \in \{0, \ldots, T - 1\}$, cioè tutti i $\Theta(T^2)$ intervalli $[\alpha, \beta]$ di ciascun nodo, per un totale di $\Theta(nT^2)$ coppie nodo–finestra nel caso peggiore. Costruire invece l'indice a query time per il μ richiesto equivale a esaminare tutte le $n(T - \mu)$ coppie nodo–tempo, e non migliora la ricerca esaustiva.

Idea: un oracolo sul primo blocco. Due signature vengono confrontate blocco per blocco. Per scartare rapidamente i nodi che non possono essere simili a v non è quindi necessario indicizzare l'intera signature: ci basta sapere se il *primo blocco* di $S_v([t, t + \mu])$ coincide con il primo blocco di $S_u([t, t + \mu])$. Quando parliamo di primo blocco intendiamo la prima *permutazione* della signature MinHash. Fissiamo il primo blocco, scrivendo $h_v = h_v^1$ e $\pi = \pi_1$. Supponiamo ora di disporre di un **oracolo** che, dati v e μ, in tempo veloce ci dice se esistono nodi $u \neq v$ con lo stesso primo blocco di v in qualche finestra $[t, t + \mu]$, senza esaminare le $n(T - \mu)$ coppie nodo–tempo. Con questo oracolo è possibile effettuare un *filtro preliminare* sui nodi, in due fasi.

1. **Filtro:** si usa l'oracolo per ottenere i nodi u (con le relative finestre t) che hanno il primo blocco uguale a quello di v.

2. **Verifica:** per ogni coppia candidata si calcolano le signature complete tramite l'oracolo $\mathcal{O}$. Il primo blocco è già uguale, quindi si confrontano i restanti $k - 1$ blocchi e si controlla che $\sigma(S_v, S_u) \geq \tau$.

Una coppia (u, t) supera il filtro con probabilità pari all'indice di Jaccard, di conseguenza una coppia, seppur simile, potrebbe anche non superarlo. Per tale motivo l'algoritmo è un'**euristica**: il filtro non è esatto, e il numero di candidati da verificare

dipende dai dati. Il sottoproblema da risolvere è dunque la costruzione dell'oracolo. Il resto della sezione descrive la sua struttura dati e il suo algoritmo di query, e infine come l'oracolo viene usato per rispondere alla query del problema.

Definizione 5.1.1 (Oracolo sul primo blocco)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ un grafo temporale e π una permutazione di V. Si vuole processare $\mathcal{G}$ in modo tale da poter rispondere alla seguente query: dati un vertice $v \in V$ e un intero $\mu \geq 0$, ritornare un vertice $u \in V \setminus \{v\}$ (se esiste) tale che esiste un $t \in [T]$ per cui vale*

$$h_v([t, t+\mu]) = h_u([t, t+\mu]) < +\infty.$$

La condizione $< +\infty$ esclude le finestre in cui nessuno dei due nodi ha vicini: altrimenti due nodi senza vicini nella finestra risulterebbero "uguali" senza alcuna similarità. È circa lo stesso principio del pruning temporale $W(v)$ della Sezione 4.2.

5.2 Rappresentazione del primo blocco tramite rettangoli

Fissiamo un nodo v e poniamo $h_v(t) = \min \pi(N_t(v))$ con $h_v(t) = +\infty$ se $N_t(v) = \emptyset$, ossia se $t \notin \Lambda(v)$. Per ogni intervallo $[\alpha, \beta]$ vale

$$h_v([\alpha, \beta]) = \min_{t \in [\alpha, \beta]} h_v(t)$$

che si calcola con spazio lineare e tempo costante tramite una Range Minimum Query (RMQ). Definiamo

$$b_v(t) = (h_v(t), t)$$

e ordiniamo i $b_v(t)$ in modo lessicografico, ovvero $b_v(t_1) < b_v(t_2)$ se $h_v(t_1) < h_v(t_2)$, oppure $h_v(t_1) = h_v(t_2)$ e $t_1 < t_2$. I punti differiscono almeno per la seconda coordinata, quindi l'ordine è totale e, per ogni intervallo, il minimo

$$p_v(\alpha, \beta) = \arg\min_{t \in [\alpha, \beta]} b_v(t)$$

è unico, con $h_v([\alpha, \beta]) = h_v(p_v(\alpha, \beta))$. Vale quindi il seguente Lemma:

Lemma 5.1. *Sia $p \in [T]$ e siano*

$$L_p = \max\{j < p : b_v(j) < b_v(p)\} \qquad \textit{(con } L_p = 0 \textit{ se l'insieme è vuoto)}$$
$$R_p = \min\{j > p : b_v(j) < b_v(p)\} \qquad \textit{(con } R_p = T + 1 \textit{ se l'insieme è vuoto)}$$

Allora

$$p_v(\alpha, \beta) = p \iff L_p < \alpha \leq p \leq \beta < R_p.$$

Dimostrazione. Per la totalità dell'ordine, $p_v(\alpha, \beta) = p$ se e solo se $p \in [\alpha, \beta]$ e nessun $j \in [\alpha, \beta]$ ha $b_v(j) < b_v(p)$. A sinistra di p: se $L_p \geq \alpha$, allora $j = L_p \in [\alpha, p)$ è un controesempio; se $L_p < \alpha$, ogni $j < p$ con $b_v(j) < b_v(p)$ soddisfa $j \leq L_p < \alpha$. Se l'insieme è vuoto, $L_p = 0 < \alpha$ poiché $\alpha \geq 1$. A destra di p si ragiona in modo simmetrico, con $R_p = T + 1 > \beta$ se l'insieme è vuoto, essendo $\beta \leq T$. □

Possiamo quindi rappresentare ogni intervallo $[\alpha, \beta]$ come il punto (α, β) del piano $[T] \times [T]$, con l'istante di inizio sull'ascissa e quello di fine sull'ordinata. Anche qui, vale il seguente Lemma:

Lemma 5.2. *Per ogni $p \in [T]$, l'insieme degli intervalli $[\alpha, \beta]$ con $p_v(\alpha, \beta) = p$ è il rettangolo*

$$\mathcal{R}_{v,p} = [L_p + 1, p] \times [p, R_p - 1].$$

Dimostrazione. Per il Lemma 5.1, l'insieme è $\{(\alpha, \beta) : L_p < \alpha \leq p \leq \beta < R_p\}$, che è il prodotto cartesiano indicato. □

Ogni intervallo ha un solo argmin, quindi i rettangoli $\mathcal{R}_{v,p}$, $p \in [T]$, sono a due a due disgiunti e ricoprono tutti gli intervalli $[\alpha, \beta]$ con $\alpha \leq \beta$. In particolare, per ogni t, il punto $(t, t + \mu)$ sta in un solo rettangolo di v. Inoltre, il Lemma 5.2 garantisce che

$$h_v([\alpha, \beta]) = h_v(p) \quad , \forall (\alpha, \beta) \in \mathcal{R}_{v,p}$$

Questa quantità viene chiamata *etichetta* del rettangolo. Un rettangolo ha etichetta finita se e solo se $p \in \Lambda(v)$. Scartiamo quindi i rettangoli con etichetta $+\infty$, che corrispondono a finestre senza vicini, e indichiamo con $\mathcal{R}(v)$ l'insieme di quelli rimanenti. Vale quindi che:

$$|\mathcal{R}(v)| \leq |\Lambda(v)| \leq T.$$

I $\Theta(T^2)$ intervalli di v sono quindi descritti da $O(T)$ rettangoli, ciascuno con la propria etichetta. Lo spazio totale occupato per tutti i vertici è $O(nT)$.

Le finestre di ampiezza μ. Data una query (v, μ), se $\mu \geq T$ non esiste alcuna finestra $[t, t + \mu] \subseteq [T]$ e la risposta è vuota. Altrimenti gli intervalli di interesse sono tutti e soli i punti $(t, t + \mu)$, cioè la retta $D_\mu : \beta = \alpha + \mu$ (tempo di fine uguale al tempo di inizio più μ). Per un rettangolo $\mathcal{R} = [\ell, p] \times [p, r]$ poniamo

$$X_{\mathcal{R}}(\mu) = \{t : (t, t + \mu) \in \mathcal{R}\} \qquad \text{e} \qquad s(\mathcal{R}) = r - \ell,$$

dove $s(\mathcal{R})$ è lo *span* del rettangolo. Avendo definito $X_{\mathcal{R}}(\mu)$ e $s(\mathcal{R})$, è possibile dimostrare il seguente lemma

Lemma 5.3. *$X_{\mathcal{R}}(\mu) = [a, b]$, con $a = \max\{\ell, p - \mu\}, b = \min\{p, r - \mu\}$, ed è non vuoto se e solo se $s(\mathcal{R}) \geq \mu$.*

Dimostrazione. Si ha $t \in X_{\mathcal{R}}(\mu)$ se e solo se $\ell \leq t \leq p$ e $p \leq t + \mu \leq r$, cioè $t \in [\ell, p] \cap [p - \mu, r - \mu]$. L'intersezione è non vuota se e solo se ogni estremo inferiore è minore o uguale a ogni estremo superiore. Le condizioni $\ell \leq p$, $p - \mu \leq p$ e $p - \mu \leq r - \mu$ valgono sempre, perché $\ell \leq p \leq r$ e $\mu \geq 0$. Resta solo $\ell \leq r - \mu$, cioè $s(\mathcal{R}) \geq \mu$. Inoltre $t \geq \ell \geq 1$ e $t + \mu \leq r \leq T$, quindi gli estremi sono automaticamente in $[T]$. □

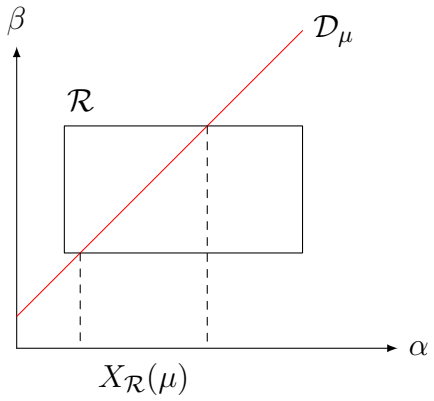


Figura 6: Rappresentazione grafica di $X_{\mathcal{R}}(\mu)$.

Dunque i rettangoli di v *rilevanti* per μ, cioè quelli che intersecano D_μ, sono tutti e soli quelli con $s(\mathcal{R}) \geq \mu$. I loro insiemi $X_{\mathcal{R}}(\mu)$ sono intervalli non vuoti e a due a due disgiunti, contenuti in $[1, T - \mu]$. È possibile quindi definire la *risposta ammissibile sulla query* (v, μ) tramite il seguente lemma.

Lemma 5.4. *Sia $u \neq v$. Allora il vertice u è una risposta ammissibile all'oracolo sulla query (v, μ) se e solo se esistono $\mathcal{R}_v \in \mathcal{R}(v)$ e $\mathcal{R}_u \in \mathcal{R}(u)$, con la stessa etichetta, tali che $X_{\mathcal{R}_v}(\mu) \cap X_{\mathcal{R}_u}(\mu) \neq \emptyset$.*

Dimostrazione. ($\Rightarrow$) Sia t tale che $h_v([t, t + \mu]) = h_u([t, t + \mu]) = x$, con x finito. Per la partizione, $(t, t + \mu)$ appartiene a un unico rettangolo $\mathcal{R}_v$ di v e a un unico $\mathcal{R}_u$ di u, le cui etichette, per il Lemma 5.2, sono rispettivamente $h_v([t, t+\mu])$ e $h_u([t, t+\mu])$. Sono quindi uguali a x, e t appartiene a entrambi gli insiemi X.
($\Leftarrow$) Se $t \in X_{\mathcal{R}_v}(\mu) \cap X_{\mathcal{R}_u}(\mu)$ allora $(t, t+\mu) \in \mathcal{R}_v \cap \mathcal{R}_u$. Per il Lemma 5.2, $h_v([t, t+\mu])$ e $h_u([t, t + \mu])$ sono le etichette dei due rettangoli, che sono uguali e finiti. □

Il sottoproblema si riduce quindi a questo: per ogni rettangolo rilevante $\mathcal{R}_v$ di v, trovare i rettangoli di altri nodi con stessa etichetta che incontrano il segmento $\mathcal{R}_v \cap D_\mu$.

Condizione di intersezione. Sia $[a, b] = X_{\mathcal{R}_v}(\mu)$ e sia $\mathcal{R}_u = [\ell_u, p_u] \times [p_u, r_u]$ un altro rettangolo con lo stesso valore. $\mathcal{R}_u$ incontra il segmento $\mathcal{R}_v \cap D_\mu$ se e solo se esiste un tempo t che soddisfa le seguenti condizioni

$$\begin{cases} a \leq t \leq b \\ \ell_u \leq t \leq p_u \\ p_u - \mu \leq t \leq r_u - \mu \end{cases} \iff \begin{cases} \ell_u \leq b \\ a \leq p_u \leq b + \mu \\ r_u \geq a + \mu \\ r_u - \ell_u \geq \mu. \end{cases}$$

Infatti il sistema equivale a $[a, b] \cap [\ell_u, p_u] \cap [p_u - \mu, r_u - \mu] \neq \emptyset$ e, per intervalli in dimensione 1, l'intersezione di tre intervalli è non vuota se e solo se lo sono a due a

due (teorema di Helly [17]). Le coppie danno: $\ell_u \le b$ e $a \le p_u$ (da $[a,b]$ e $[\ell_u, p_u]$); $p_u - \mu \le b$ e $a \le r_u - \mu$ (da $[a,b]$ e $[p_u - \mu, r_u - \mu]$); $\ell_u \le r_u - \mu$ e $p_u - \mu \le p_u$ (da $[\ell_u, p_u]$ e $[p_u - \mu, r_u - \mu]$), quest'ultima sempre vera. Le restanti condizioni sui singoli intervalli sono verificate per costruzione. Possiamo quindi rappresentare ogni rettangolo $\mathcal{R} = [\ell, p] \times [p, r]$ come un punto in quattro dimensioni, di coordinate $P(\mathcal{R}) = (\ell, p, r, r - \ell)$. I rettangoli che incontrano il segmento $\mathcal{R}_v \cap D_\mu$ si ritrovano con una *orthogonal range query* del tipo

$$\boxed{P(\mathcal{R}) \in (-\infty, b] \times [a, b + \mu] \times [a + \mu, +\infty) \times [\mu, +\infty).}$$

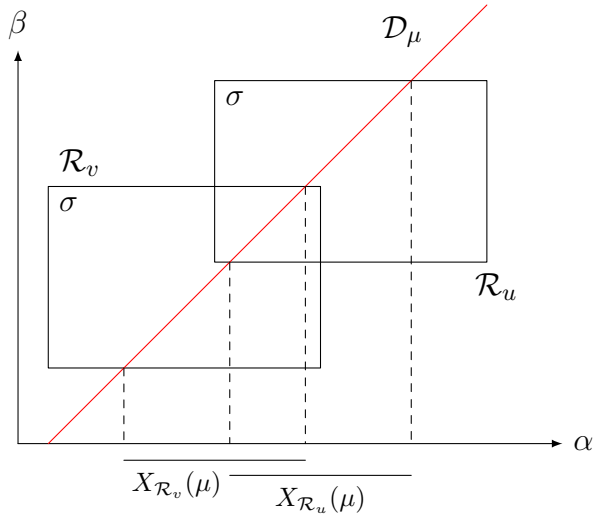


Figura 7: Rappresentazione grafica di $X_{\mathcal{R}_v}(\mu) \cap X_{\mathcal{R}_u}(\mu)$.

5.3 Algoritmo di generazione dell'indice

Sia $N \le nT$ il numero totale di rettangoli, con etichetta finita, di tutti i nodi. L'indice è composto da tre strutture.

Lista dei rettangoli di ogni nodo. Per ogni nodo v si salva la lista $\mathcal{L}(v)$ dei record $(\ell, p, r, x) = (L_p + 1, p, R_p - 1, h_v(p))$, uno per ogni rettangolo di $\mathcal{R}(v)$, ordinata per span $s = r - \ell$ *decrescente*. In questo modo i rettangoli rilevanti per μ, cioè quelli con $s \ge \mu$, formano un prefisso della lista. Spazio: $O(T)$ per nodo, $O(nT)$ in totale.

Dizionario delle etichette. Le etichette sono ranghi $\pi(u) \in [n]$. Come per i dizionari H_j delle sezioni precedenti, l'indice contiene un dizionario H indicizzato dall'etichetta x, tale che $H[x]$ punta alla struttura $\mathcal{D}_x$ (ed è vuoto se nessun rettangolo ha etichetta x). L'accesso costa $O(1)$.

Strutture per range query. Per ogni valore x sia $\mathcal{Q}_x$ l'insieme dei rettangoli di *tutti* i nodi con valore x. A ciascun $\mathcal{R} \in \mathcal{Q}_x$ associamo il punto $P(\mathcal{R}) = (\ell, p, r, s)$, con $s = r - \ell$, insieme al nodo v a cui appartiene. La struttura $\mathcal{D}_x$ è un **range tree a quattro livelli** (introdotto e formalizzato qui [18]) sui punti $P(\mathcal{R})$, $\mathcal{R} \in \mathcal{Q}_x$, in cui ciascun livello corrisponde a una delle quattro condizioni della range query:

- **Livello 1**: un albero binario di ricerca bilanciato sulla coordinata ℓ.
- **Livello 2**: ogni nodo del livello 1 contiene un albero bilanciato sulla coordinata p dei punti del proprio sottoalbero.
- **Livello 3**: ogni nodo del livello 2 contiene un albero bilanciato sulla coordinata r dei punti del proprio sottoalbero.
- **Livello 4**: ogni nodo del livello 3 contiene la lista, ordinata per span s decrescente, dei punti del proprio sottoalbero.

Ogni livello moltiplica lo spazio per $\log N$, e l'ultimo livello è una lista ordinata da cui si estraggono i punti con span $\geq \mu$. Nella figura sottostante è possibile vedere un'esempio di *Range Tree 4D*, ovviamente non completo. Si mostrano infatti solamente i livelli relaviti alla *radice* di ogni albero

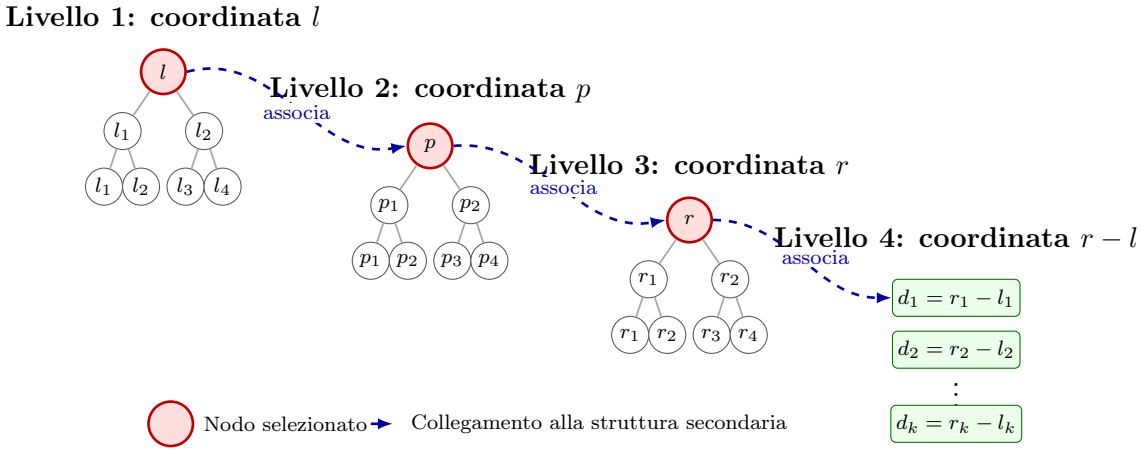


Figura 8: Esempio RangeTree 4D.

Recupero dei punti nel box. Data la query (a, b, μ) su $\mathcal{D}_x$ e un intero q, l'operazione $\textsc{Report}(\mathcal{D}_x; a, b, \mu; q)$ restituisce al più q punti che soddisfano le quattro condizioni del box. Essa procede come segue.

1. Il prefisso $\ell \leq b$ viene scomposto in $O(\log N)$ nodi canonici del livello 1; in ciascuno, l'intervallo $p \in [a, b + \mu]$ viene scomposto in $O(\log N)$ nodi canonici

del livello 2; in ciascuno, il suffisso $r \geq a + \mu$ viene scomposto in $O(\log N)$ nodi canonici del livello 3. Si ottengono $O(\log^3 N)$ nodi canonici del livello 3, i cui insiemi di punti sono a due a due disgiunti.

2. Per ogni nodo canonico si scorre la lista del livello 4 dal primo punto, aggiungendo all'output i punti con span $s \geq \mu$ e interrompendo la lista al primo punto con $s < \mu$.

3. Ci si ferma appena sono stati raccolti q punti. Se i nodi finiscono prima, si restituiscono quelli raccolti.

Con $q = 2$ il costo è $O(\log^3 N)$. Con $q = \infty$ l'operazione riporta tutti i punti del box, e il costo è $O(\log^3 N + |PR|)$, con $|PR|$ pari al numeri di punti riportati.

Pseudocodice della generazione dell'indice. Per ogni nodo v, i valori L_p e R_p si calcolano con due passate di una pila monotona rispetto all'ordine totale $b_v(\cdot)$ sugli istanti $t = 1, \ldots, T$: una passata da sinistra per L_p e una da destra per R_p.

Algorithm 3 Costruzione dell'indice per l'oracolo sul primo blocco

Require: Graph $\mathcal{G} = (V, E, \lambda)$, permutazione π
Ensure: Liste $\mathcal{L}(v)$, Dizionario H

```
1:  function BuildIndex3(G)
2:      h_v(t) ← +∞ per ogni v ∈ V, t ∈ [T]
3:      for all e = {v, u} ∈ E e t ∈ λ(e) do
4:          h_v(t) ← min{h_v(t), π(u)},    h_u(t) ← min{h_u(t), π(v)}
5:      end for
6:      Q_x ← ∅ per ogni x
7:      for all v ∈ V do
8:          Calcola L_p, R_p per p ∈ [T]
9:          for all p ∈ [T] with h_v(p) < +∞ do
10:             L(v).append((L_p + 1, p, R_p − 1, h_v(p)))
11:             Q_{h_v(p)}.insert(P(R_{v,p}), v)
12:         end for
13:         Ordina L(v) secondo span decrescente
14:     end for
15:     for all x con Q_x ≠ ∅ do
16:         H[x] ← BuildRangeTree4D(Q_x)
17:     end for
18:     return L, H
19: end function
```

In Linea 4 si calcolano i valori $h_v(t)$. In Linea 8 si ottengono i rettangoli di v in tempo $O(T)$, scartando quelli con etichetta $+\infty$. In Linea 16 si costruiscono le strutture $\mathcal{D}_x$

in modo standard, ordinando i punti per ℓ e costruendo i livelli successivi per fusione di liste ordinate.

5.4 Algoritmo di risposta alla query

Per una query (v, μ) l'algoritmo restituisce subito la risposta vuota se $\mu \geq T$. Altrimenti scorre la lista $\mathcal{L}(v)$ dal rettangolo con span maggiore. Per ciascun rettangolo $\mathcal{R}_v = [\ell_v, p_v] \times [p_v, r_v]$ con etichetta x e span $s(\mathcal{R}_v) \geq \mu$ si calcola $[a, b] = X_{\mathcal{R}_v}(\mu)$ con il Lemma 5.3, cioè

$$a = \max\{\ell_v, p_v - \mu\}, \qquad b = \min\{p_v, r_v - \mu\},$$

e si interroga $\mathcal{D}_x = H[x]$ con il box $(-\infty, b] \times [a, b + \mu] \times [a + \mu, +\infty) \times [\mu, +\infty)$. Se si incontra un rettangolo con $r_v - \ell_v < \mu$ la scansione si interrompe: gli span sono decrescenti, quindi anche i rettangoli successivi non intersecano D_μ (Lemma 5.3). Ogni iterazione fissa il rettangolo $\mathcal{R}_v$ di v che contiene i punti $(t, t + \mu)$ per $t \in [a, b]$, cioè le finestre in cui l'etichetta di v è x, e chiede se esiste un rettangolo di un altro nodo, con la stessa etichetta x, che contiene un punto $(t, t + \mu)$ con $t \in [a, b]$. Per il Lemma 5.4 la risposta è sì se e solo se esiste un nodo u ammissibile con questa coppia di rettangoli. Il ciclo sui rettangoli è necessario, perché la risposta può venire da una qualsiasi finestra t: per la partizione, ogni $t \in [1, T - \mu]$ cade in un solo rettangolo di v, quindi il ciclo copre tutte le finestre possibili.

Uso come filtro preliminare (tutti i candidati). Per rispondere alla query del problema servono tutti i nodi con lo stesso primo blocco di v, e non uno solo. Si usa quindi $q = \infty$: per ogni punto riportato, appartenente a un rettangolo $\mathcal{R}_u$ di un nodo $u \neq v$, l'insieme dei suoi istanti candidati è

$$I_u = [a, b] \cap X_{\mathcal{R}_u}(\mu),$$

non vuoto per la condizione di intersezione. Per il Lemma 5.4, l'insieme dei candidati prodotti da tutti i rettangoli rilevanti è

$$C(v, \mu) = \big\{(u, t) : u \neq v,\ t \in [T - \mu],\ h_v([t, t + \mu]) = h_u([t, t + \mu]) < +\infty\big\}.$$

Ogni coppia compare una volta sola, perché i rettangoli di u sono disgiunti e quindi gli insiemi I_u emessi per uno stesso u sono disgiunti. Per ogni $(u, t) \in C(v, \mu)$ si esegue poi la verifica: si recuperano $S_v([t, t + \mu])$ e $S_u([t, t + \mu])$ dalle strutture $\mathcal{O}_v$ e $\mathcal{O}_u$ e, se

$$SIM = \sigma\big(S_v([t, t + \mu]), S_u([t, t + \mu])\big) \geq \tau,$$

la query restituisce u e l'algoritmo ritorna **True**. Se i candidati si esauriscono senza validazioni positive, restituisce **False**. Poiché la verifica usa le signature complete, l'algoritmo non produce falsi positivi; può invece mancare una coppia simile se questa non supera il filtro sul primo blocco.

Algorithm 4 Query3

Require: Grafo $\mathcal{G} = (V, E, \lambda)$, τ, Nodo $v \in V$, $\mu \geq 0$
Ensure: True se un candidato valido esiste, False altrimenti

```
1: function QUERY3(v, μ)
2:     if μ ≥ T then
3:         return False
4:     end if
5:     for all (ℓ_v, p_v, r_v, x) ∈ L(v) ord. per span decrescente do
6:         if r_v − ℓ_v < μ then
7:             break                    ▷ I prossimi rettangoli non incontreranno D_μ
8:         end if
9:         a ← max{ℓ_v, p_v − μ},   b ← min{p_v, r_v − μ}
10:        for all (R_u, u) ∈ REPORT(H[x]; a, b, μ; ∞) con u ≠ v do
11:            for all t ∈ [a, b] ∩ X_{R_u}(μ) do
12:                S_v ← O_v.query([t, t + μ]),   S_u ← O_u.query([t, t + μ])
13:                if σ(S_v, S_u) ≥ τ then
14:                    return True
15:                end if
16:            end for
17:        end for
18:    end for
19:    return False
20: end function
```

Pseudocodice dell'algoritmo di query L'algoritmo 4 fornisce lo pseudocodice dettagliato per rispondere alla query
Nella prossima sezione analizzeremo la complessità riguardante la costruzione di questo oracolo, e della risposta alla query. A tal scopo, possiamo fornire il seguente teorema, che preannuncia i bound teorici ottenuti.

Teorema 5.5 (Bound asintotici)**.** *Sia $\mathcal{G} = (V, E, \lambda)$ il grafo temporale in input e $\tau \in [0, 1]$ il valore soglia. L'oracolo proposto*

- *viene costruito in tempo $O\Big(\sum_e |\lambda(e)| + nT + N \log^3 N\Big) \approx O(|\mathcal{E}| + N \log^3 N)$*
- *utilizza spazio ausiliare pari a $O\big(nT + N \log^3 N\big) \subseteq O\big(nT \log^3(nT)\big)$*
- *permette di rispondere ad una query in tempo $O\big(k_v(\mu) \log^3 N + M\big) \approx O(T \log^3 nT + M)$, con $M = |C(v, \mu)|$ = numero candidati*

5.5 Analisi della complessità

Complessità della costruzione dell'indice
Il calcolo dei valori $h_v(t)$ costa $O(\sum_e |\lambda(e)| + nT)$. Per ogni nodo, il calcolo di L_p e R_p costa $O(T)$, e l'ordinamento di $\mathcal{L}(v)$ costa $O(T \log T)$. La distribuzione degli N rettangoli negli insiemi $\mathcal{Q}_x$ costa $O(N + n)$. Infine, ogni livello del range tree aggiunge un fattore $\log N_x$, con $N_x = |\mathcal{Q}_x|$, quindi la costruzione di $\mathcal{D}_x$ richiede tempo $O(N_x \log^3 N_x)$; sommando su x si ottiene $O(N \log^3 N)$. Il tempo complessivo è quindi

$$O\Big(\sum_e |\lambda(e)| + nT + N \log^3 N\Big) \approx O(|\mathcal{E}| + N \log^3 N)$$

Per quanto riguarda lo spazio, le liste $\mathcal{L}(v)$ e il dizionario H occupano $O(nT)$, mentre le strutture $\mathcal{D}_x$ occupano $O(N \log^3 N)$, per un totale di

$$O\big(nT + N \log^3 N\big) \subseteq O\big(nT \log^3(nT)\big).$$

Va aggiunto lo spazio utilizzato dall'oracolo $\mathcal{O}$ per il recupero dello sketch completo, che non dipende da questo indice.

Complessità della query Sia

$$k_v(\mu) = |\{\mathcal{R} \in \mathcal{R}(v) : s(\mathcal{R}) \geq \mu\}| \leq \min\{T, T - \mu\}$$

il numero di rettangoli rilevanti, cioè il numero di iterazioni del ciclo esterno: per la partizione, gli insiemi $X_{\mathcal{R}}(\mu)$ dei rettangoli rilevanti sono non vuoti, a due a due disgiunti e contenuti in $[1, T - \mu]$. Ogni iterazione costa $O(\log^3 N)$, per un accesso a H e una chiamata a REPORT. Con il fractional cascading [19] si può ridurre di un fattore log il tempo di ciascuna chiamata a REPORT. Usato come filtro, con $q = \infty$, sia $M = |C(v, \mu)| = \sum_u |I_u|$ il numero di coppie candidate. Il numero di punti riportati è al più $k_v(\mu) + M$, perché ogni punto riportato, tranne $\mathcal{R}_v$, contiene almeno un candidato. Il costo della fase di filtro è quindi

$$O\big(k_v(\mu) \log^3 N + M\big) \approx O(T \log^3 nT + M)$$

Per ogni candidato (u, t), la verifica richiede due interrogazioni all'oracolo $\mathcal{O}$, una per v e l'altra per u, entrambe di costo rispettivamente $O(k \log |\Lambda(v)|)$ e $O(k \log |\Lambda(u)|)$, e un confronto $O(k)$. La complessità della query completa di check finale risulta essere:

$$O\Big(k_v(\mu) \log^3 N + k \sum_{(u,t) \in C(v,\mu)} \big(\log |\Lambda(v)| + \log |\Lambda(u)|\big)\Big).$$

6 Esperimenti

In questo capitolo si presentano i risultati della valutazione sperimentale dei concetti introdotti nella tesi. Gli esperimenti sono tre, ciascuno dedicato a una diversa componente del lavoro:

- **Esperimento n. 1** (Sezione 6.1): valuta l'oracolo $\mathcal{O}$ per il recupero degli sketch, confrontando il tempo medio di query al variare dell'ampiezza μ, con quello della generazione naïve degli sketch.
- **Esperimento n. 2** (Sezione 6.2): riguarda la prima variante del problema. Contiene benchmark e confronti con l'algoritmo Brute Force, con l'alternativa basata su $T - \mu$ indici LSH indipendenti e con la variante priva di pruning temporale, in termini di tempi di query, costi di costruzione, memoria ausiliaria e completezza delle risposte.
- **Esperimento n. 3** (Sezione 6.3): riguarda la seconda variante del problema. Si implementa l'oracolo sul primo blocco MinHash e si analizza il numero di nodi candidati prodotti dal filtro preliminare al variare di μ.

6.1 Valutazione sperimentale dell'oracolo $\mathcal{O}$ per il recupero degli sketch

L'esperimento valuta l'efficienza dell'oracolo $\mathcal{O}$, introdotto nelle sezioni precedenti, nel recuperare lo sketch MinHash del vicinato temporale di un nodo v su un intervallo I. In particolare, il tempo di interrogazione dell'oracolo viene confrontato con quello di una procedura *naïve* che ricostruisce lo sketch a ogni query, senza sfruttare un indice temporale. Il confronto considera diverse dimensioni del grafo e ampiezze temporali μ.

6.1.1 Configurazione sperimentale

Sono stati utilizzati due modelli di grafi temporali sintetici:

- **Erdős–Rényi con interazioni ripetute:** il grafo statico $G(n, p)$ ha probabilità di connessione $p = \delta/(n-1)$, dove δ è il grado medio desiderato. A ciascun arco sono associati L eventi, con L campionato uniformemente tra 1 e n^c $(0 < c \leq 1)$, e timestamp estratti nell'orizzonte $[1, T_{\max}]$, con $T_{\max} = 30\lceil n^c \rceil$.
- **Grafo sintetico a interazioni uniformi:** vengono generati $m = n\delta$ archi temporali orientati, con estremi e timestamp campionati casualmente; questo modello permette di valutare il comportamento su reti di dimensioni maggiori.

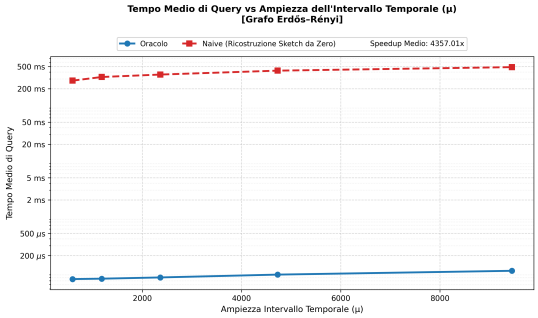
Per ciascun valore di μ sono state eseguite **100 query temporali casuali**, misurando il tempo medio necessario a restituire lo sketch.

6.1.2 Strategie a confronto

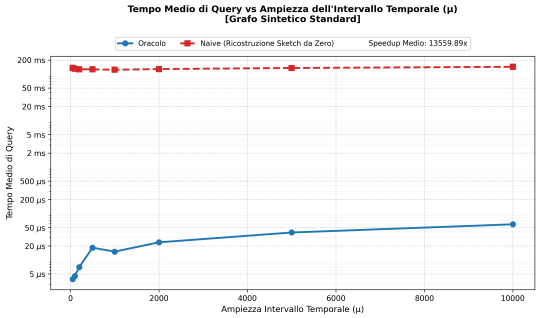
L'**oracolo** $\mathcal{O}$ restituisce lo sketch dell'intervallo richiesto sfruttando la struttura pre-indicizzata, con costo teorico di query $O(k \log |\Lambda(v)|)$, indipendente dall'ampiezza μ a parità degli altri parametri. La **ricostruzione naïve**, invece, esamina gli eventi incidenti su v e ricalcola la signature sui vicini distinti attivi in I; il relativo costo è $O(|E_v| + k|N^I(v)|)$, dove E_v è l'insieme degli eventi incidenti su v. Il benchmark confronta esclusivamente i tempi di recupero dello sketch, non i costi di costruzione degli indici.

6.1.3 Risultati e discussione

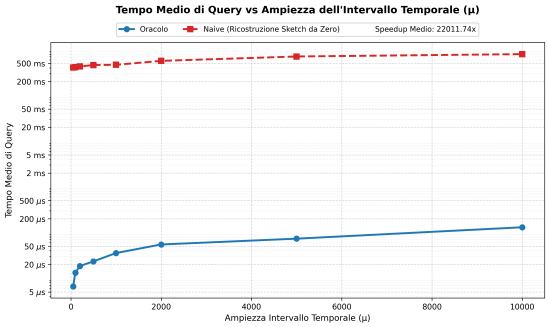
La Figura 9 riunisce i tre scenari di benchmark. I tempi di interrogazione dell'oracolo rimangono nell'ordine dei microsecondi al variare di μ: sotto i 200 μs sul grafo Erdős–Rényi e 4–60 μs sui grafi sintetici di dimensione maggiore. La ricostruzione naïve richiede invece circa 200–500 ms nel primo scenario e tempi dell'ordine di 100–700 ms negli altri due.



(a) Erdős–Rényi ($n = 10\,000$)



(b) Sintetico standard ($n = 300\,000$)



(c) Sintetico standard ($n = 70\,000$)

Figura 9: Tempo medio di recupero dello sketch al variare di μ: confronto fra l'oracolo $\mathcal{O}$ e la ricostruzione naïve. (a) Grafo Erdős–Rényi con $\delta = 32$, $k = 128$; (b) grafo sintetico standard con $\delta = 200$, $k = 64$; (c) grafo sintetico standard con $\delta = 350$, $k = 128$.

6.2 Valutazione sperimentale: Neighborhood Search con finestre di ampiezza fissa

Si valutano la riduzione delle finestre ottenuta tramite $W(v)$, i costi computazionali delle strategie di ricerca e la completezza delle risposte rispetto alla Brute Force (BF). La definizione di $W(v)$ e il funzionamento dell'oracolo $\mathcal{O}$ per il recupero degli sketch sono già stati presentati nella Sezione 4 e non vengono ripetuti.

6.2.1 Configurazione sperimentale

I dataset temporali sono stati preliminarmente discretizzati mediante `discretize_ts.cpp`. Il programma suddivide l'intervallo dei timestamp originali in bin di durata uniforme, con un obiettivo di 8000 intervalli, senza alterare l'ordine degli eventi:

$$\Delta = \max\left\{1, \left\lceil \frac{t_{\max} - t_{\min}}{8000} \right\rceil\right\}, \qquad t' = 1 + \left\lfloor \frac{t - t_{\min}}{\Delta} \right\rfloor. \tag{5}$$

Le successive misure utilizzano soltanto dataset già discretizzati. Lo script `run_all.sh` compila ed esegue in sequenza il benchmark sui dataset, eseguendo anche la discretizzazione preliminare e generando tabelle e grafici. Gli archi vengono trattati come non orientati, anche nei dataset originariamente diretti. Per tutti i dataset sono stati fissati $\tau = 0.35$, $k = 256$ componenti MinHash, $b = 8$ bande da $r = 32$ componenti e un massimo di $Q = 8000$ query su sorgenti distinte, selezionate senza reinserimento con seme 42. Quando $|V| < Q$, vengono interrogati tutti i nodi. L'ampiezza μ varia per dataset, come riportato nella Tabella 1. Le quattro strategie utilizzano lo stesso oracolo $\mathcal{O}$: *BF*, che verifica esaustivamente le finestre e i nodi; *LSH+W*, la soluzione proposta con indice LSH unificato e pruning; *LSH–W*, la variante dello stesso indice senza pruning temporale; e *Alt+W*, la soluzione con indici LSH distinti per finestra che utilizza anch'essa $W(v)$. Ogni query restituisce un valore booleano e si arresta al primo riscontro positivo. Le misure di costruzione e memoria riguardano le strutture ausiliarie delle singole strategie, escludendo l'oracolo $\mathcal{O}$ condiviso.

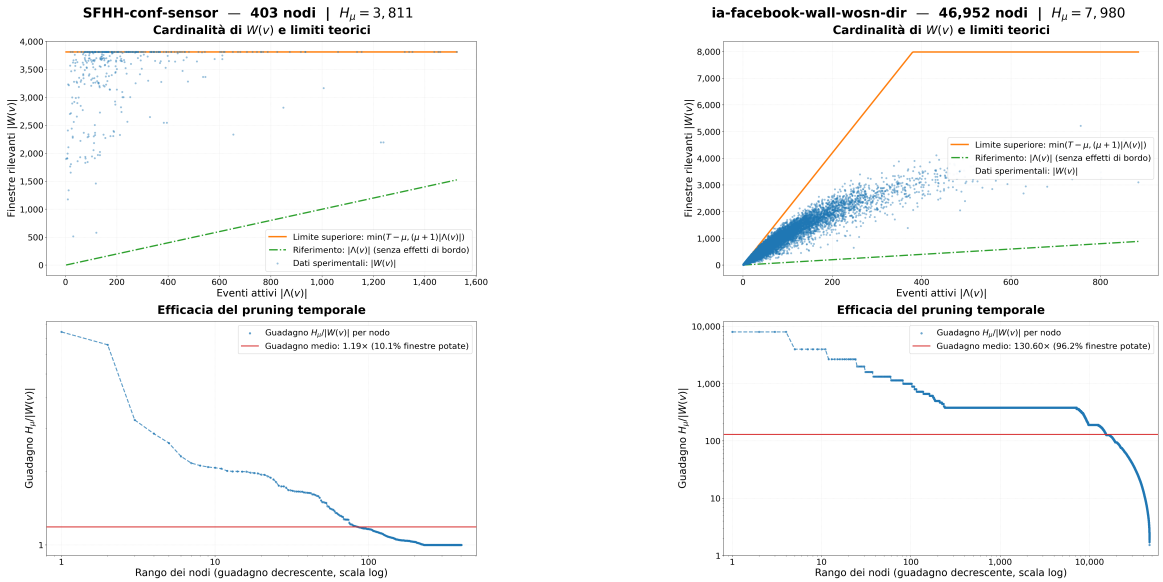
6.2.2 Pruning temporale

La Tabella 1 confronta, per tutti i dataset considerati, il numero $H_\mu = T_{\max} - T_{\min} - \mu + 1$ di inizi di finestra ammissibili con le statistiche di pruning osservate sperimentalmente. I risultati mostrano comportamenti differenti a seconda della distribuzione temporale degli eventi. In particolare, il pruning risulta modesto in `SFHH-conf-sensor`, dove viene eliminato soltanto il 10.1% delle finestre, mentre è significativamente più efficace in `ia-enron-email-dynamic`, con il 98.2% delle finestre potate, e in `ia-facebook-wall-wosn-dir`, con il 96.2%. Anche il dataset `CollegeMsg` mostra un pruning moderato (37.2%), così come `chess_year` (33.2%).
Il fattore di guadagno medio, calcolato come $H_\mu/|W(v)|$, conferma questo andamento: esso è contenuto in `SFHH-conf-sensor` (1.19×), moderato in `CollegeMsg` (2.92×)

e `chess_year` (2.04×), mentre diventa molto elevato in `ia-enron-email-dynamic` (77.41×) e soprattutto in `ia-facebook-wall-wosn-dir` (130.60×). Per evitare una rappresentazione eccessivamente affollata, nelle figure successive vengono mostrati soltanto due dataset rappresentativi: `SFHH-conf-sensor` e `ia-facebook-wall-wosn-dir`. Essi evidenziano rispettivamente un caso di pruning poco efficace e un caso in cui la riduzione delle finestre rilevanti è invece molto marcata.

Dataset	Nodi	μ	H_μ	Guadagno	Finestre potate
`chess_year`	7301	4	5	2.04×	33.2%
`CollegeMsg`	1899	3988	4009	2.92×	37.2%
`SFHH-conf-sensor`	403	3810	3811	1.19×	10.1%
`ia-enron-email-dynamic`	87273	100	7900	77.41×	98.2%
`ia-facebook-wall-wosn-dir`	46952	20	7980	130.60×	96.2%

Tabella 1: Parametri temporali e risultati del pruning per tutti i dataset considerati. Il guadagno rappresenta la media dei fattori $H_\mu/|W(v)|$ riportati nei grafici.



(a) `SFHH-conf-sensor` **(b)** `Enron`

Figura 10: Analisi della cardinalità di $W(v)$ e del fattore di guadagno del pruning temporale per i dataset `SFHH-conf-sensor` (a) e `ia-facebook-wall-wosn-dir` (b).

6.2.3 Prestazioni e memoria ausiliaria

La Tabella 2 riporta i tempi medi di interrogazione; la Tabella 3 raggruppa tempi di costruzione e memoria per le tre varianti LSH. Rispetto alla BF, LSH+W registra

speedup di circa 432× su CollegeMsg, 200× su SFHH, 1596× su chess, 8431× su Enron e 4287× su Facebook. Il confronto a parità di pruning con Alt+W favorisce l'indice unificato in tutti i dataset (1.33–3.53×); la variante LSH–W conferma invece l'utilità del pruning soprattutto su Enron, dove la query passa da 91.88 a 9.82 μs e la costruzione da 20.91 a 5.25 s. Su SFHH la variante senza pruning è marginalmente più rapida in query, perciò il vantaggio non è universale.

Dataset	**BF**	**LSH+W**	**LSH–W**	**Alt+W**
`CollegeMsg`	537579.00	**1244.46**	1468.48	2218.39
`SFHH-conf-sensor`	396216.00	1976.94	**1956.51**	2624.59
`chess_year`	3770.50	**2.36**	2.48	3.93
`ia-enron-email-dynamic`	82757.90	**9.82**	91.88	34.64
`ia-facebook-wall-wosn-dir`	165842.00	**38.69**	86.71	85.61

Tabella 2: Tempo medio di query (μs): BF e tre strategie LSH.

	Costruzione (s)			**Memoria (MiB)**		
Dataset	**LSH+W**	**LSH–W**	**Alt+W**	**LSH+W**	**LSH–W**	**Alt+W**
`CollegeMsg`	1.986	2.227	1.994	501.88	501.85	438.01
`SFHH-conf-sensor`	0.681	0.700	0.610	142.40	142.39	126.52
`chess_year`	0.034	0.033	0.040	2.63	2.51	2.38
`ia-enron-email-dynamic`	5.251	20.909	6.991	1280.08	1278.55	1152.32
`ia-facebook-wall-wosn-dir`	9.099	21.824	9.643	1447.03	1439.37	1319.27

Tabella 3: Costruzione (s) e memoria ausiliaria (MiB) degli indici LSH. Per BF, costruzione e memoria specifiche sono trascurabili; l'oracolo condiviso è escluso per tutte le strategie.

L'alternativa a indici per finestra occupa meno memoria ausiliaria, mentre l'indice unificato riduce il tempo di query. Il costo di costruzione dell'indice proposta è superiore a quello della BF, che non costruisce indici LSH; il valore nullo della sua memoria ausiliaria non rappresenta il consumo complessivo del programma. La Figura 11 illustra entrambi i confronti.

6.2.4 Completezza delle risposte rispetto alla Brute Force

Per ogni sorgente interrogata sono confrontate le risposte booleane prodotte da BF e LSH+W, utilizzando gli stessi sketch restituiti da $\mathcal{O}$ e la medesima soglia τ. Indicando con B e L gli insiemi di sorgenti per le quali BF e LSH restituiscono `True`, rispettivamente, si misura

$$\mathrm{Recall}_{\mathrm{LSH|BF}} = \frac{|B \cap L|}{|B|}, \qquad |B| > 0. \tag{6}$$

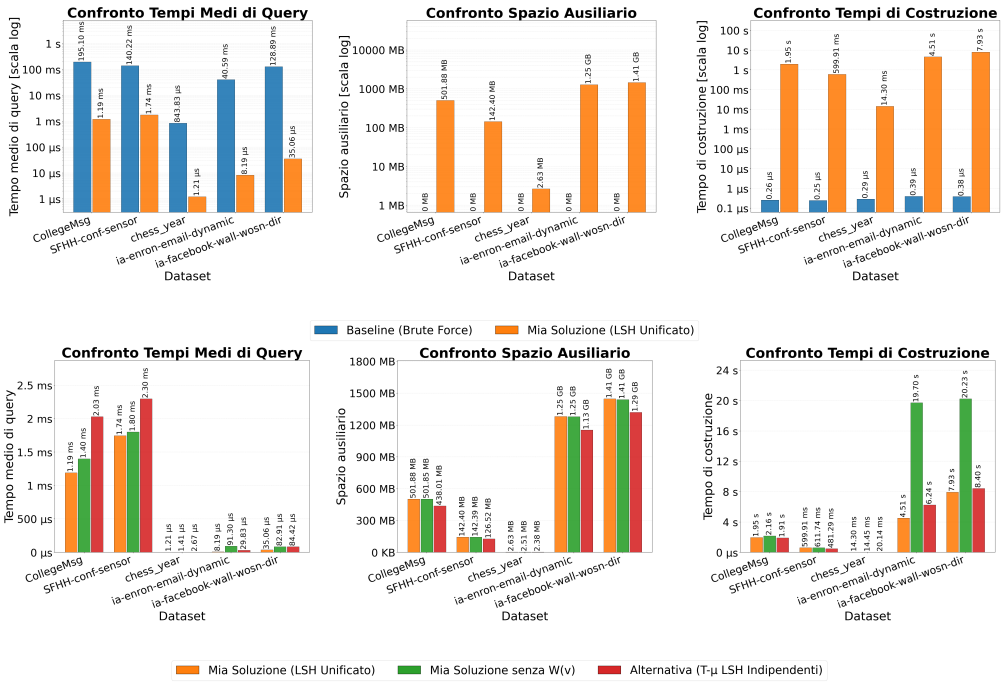


Figura 11: Tempi medi di query, memoria ausiliaria e costruzione: BF contro LSH+W (in alto, scale logaritmiche) e confronto tra LSH+W, LSH–W e Alt+W (in basso, scale lineari).

La BF è esaustiva rispetto al criterio di similarità *stimato* con le signature MinHash, non rispetto al Jaccard esatto sui vicinati.

Dataset	Q	$\lvert B\rvert$	$\lvert L\rvert$	FN	Recall
`chess_year`	7301	3115	379	2736	12.17%
`CollegeMsg`	1899	1615	1087	528	67.31%
`SFHH-conf-sensor`	403	358	111	247	31.01%
`ia-enron-email-dynamic`	8000	7715	7566	149	98.07%
`ia-facebook-wall-wosn-dir`	8000	6363	5938	425	93.32%

Tabella 4: Completezza delle risposte positive della LSH rispetto alla BF

La recall varia sensibilmente con il dataset: 98.07% su Enron e 93.32% su Facebook, ma solo 31.01% su SFHH e 12.17% su chess. In quest'ultimo caso la LSH perde 2736 delle 3115 risposte positive della BF. I falsi positivi rispetto alla BF sono nulli in tutti i campioni; i falsi negativi derivano dal filtraggio approssimato dei candidati tramite bande LSH. La Figura 12 visualizza il contrasto tra chess ed Enron.

6.2.5 Sintesi dei risultati

Le misure confermano che il pruning temporale e l'indicizzazione unificata possono ridurre sensibilmente tempi di costruzione e query, con benefici particolarmente marcati sui grafi Enron e Facebook. Il vantaggio dipende tuttavia dal dataset e richiede memoria ausiliaria e preprocessing. Soprattutto, lo speedup non implica completezza: la recall, misurata rispetto alla ricerca esaustiva sugli stessi sketch, varia dal 12.17% al 98.07%. Le prestazioni vanno quindi interpretate insieme ai falsi negativi introdotti dal filtro LSH.

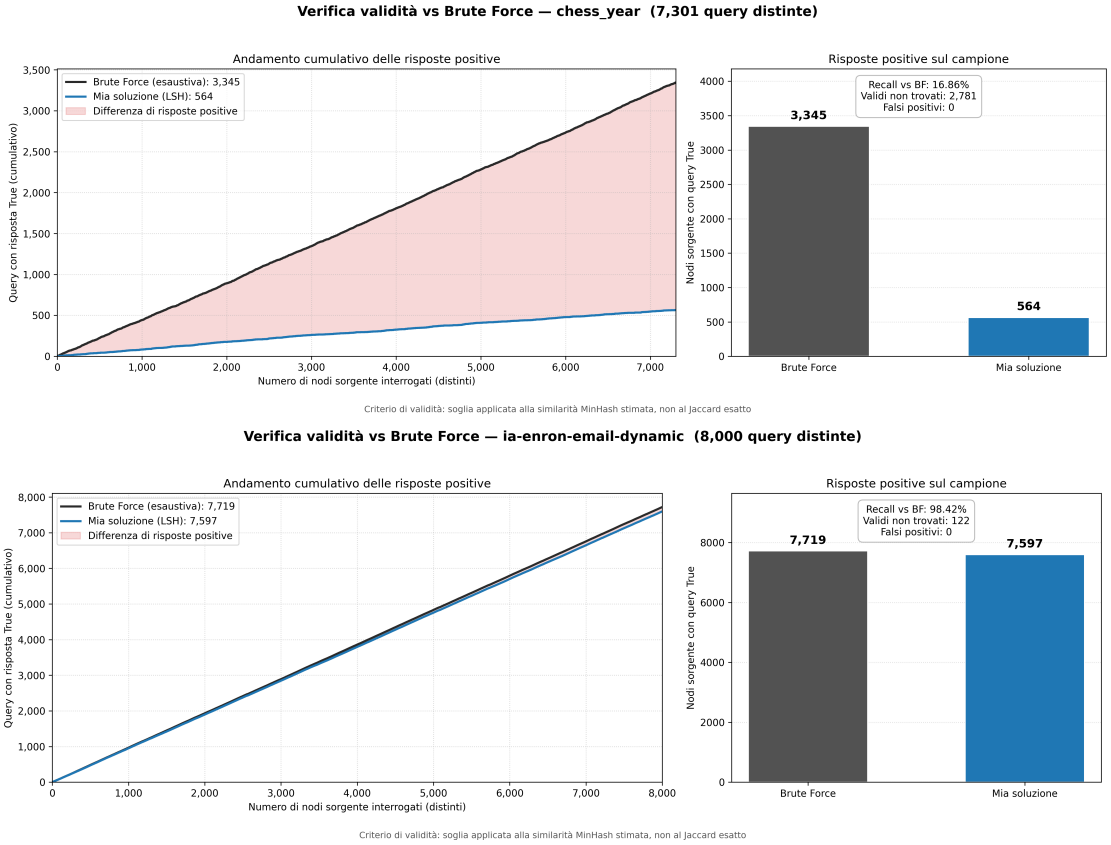


Figura 12: Risposte positive cumulative e confronto finale fra BF e LSH+W: chess (in alto) ed Enron (in basso).

6.3 Valutazione sperimentale del filtro preliminare dell'oracolo $\mathcal{O}$

L'esperimento valuta la selettività del filtro preliminare basato sulla prima componente MinHash, studiando come varia il numero di nodi candidati al variare dell'ampiezza temporale μ. Indicando con $C_1(v, \mu)$ l'insieme dei nodi distinti $u \neq v$ che superano il

filtro per almeno una finestra temporale ammissibile, si misura $|C_1(v, \mu)|$. La successiva verifica mediante la signature MinHash completa non viene eseguita: i risultati descrivono quindi il solo filtro preliminare, non la completezza delle risposte finali.

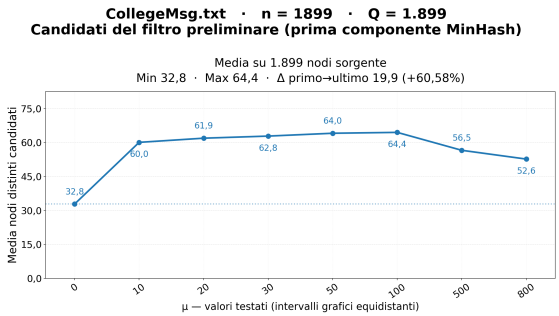
Configurazione sperimentale. I dataset sono stati preventivamente discretizzati mediante `discretize_ts.cpp`, utilizzando bin temporali uniformi con target pari a 8000. Lo script `run_all.sh` esegue la discretizzazione, costruisce una sola volta per dataset l'indice dell'oracolo $\mathcal{O}$ e ripete le interrogazioni per i valori di μ indicati nella Tabella 5. La configurazione riportata prevede `NUM_QUERIES=0`, dunque vengono interrogati *tutti* i nodi del grafo, senza reinserimento; per ciascun dataset si riutilizzano le stesse sorgenti per ogni μ. Il seme pseudo-casuale è 42 e il limite configurato per l'indice è 6144 MiB.

Tabella 5: Dataset, numero di sorgenti interrogate e ampiezze temporali considerate (espresse in unità discretizzate).

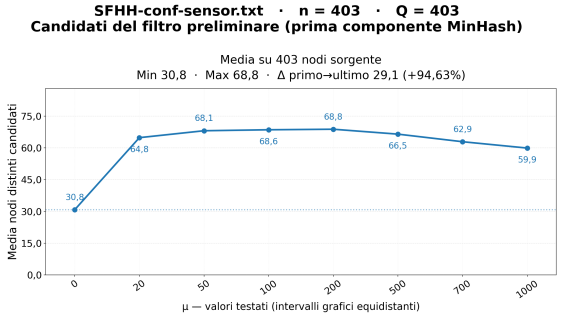
Dataset	$\|V\| = Q$	μ
`chess_year`	7301	$0, 1, 2, 4, 7, 8$
`CollegeMsg`	1899	$0, 10, 20, 30, 50, 100, 500, 800$
`SFHH-conf-sensor`	403	$0, 20, 50, 100, 200, 500, 700, 1000$
`ia-enron-email-dynamic`	87273	$0, 10, 20, 40, 100, 200, 500, 700$
`ia-facebook-wall-wosn-dir`	46952	$0, 50, 100, 200, 500, 1000, 2500, 5000$

Per ogni dataset si riportano, rispettivamente, i candidati relativi a una sorgente selezionata casualmente e la media sulle Q sorgenti, $Q^{-1} \sum_{v \in V} |C_1(v, \mu)|$. I nodi ripetuti in finestre multiple vengono conteggiati una sola volta: la metrica mostrata nei grafici non coincide con il numero di coppie candidate (u, t), pur registrato separatamente dal programma. Il punto $\mu = 0$ corrisponde al confronto su finestre di durata temporale nulla.

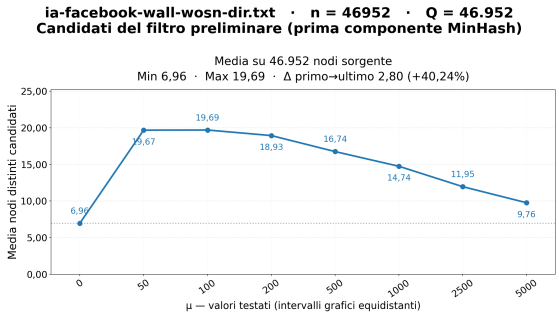
Risultati. Le Figura 13 mostrano comportamenti differenti. In `chess_year` il numero medio rimane contenuto (circa 10–15 candidati), con un massimo intorno a $\mu = 2$. In `CollegeMsg` e `SFHH-conf-sensor` la media cresce inizialmente, raggiunge valori prossimi a 65–70 e diminuisce per le finestre più ampie. Il dataset Facebook presenta un picco di circa 20 candidati medi per $\mu = 50$–100, seguito da una diminuzione progressiva fino a circa 10 per $\mu = 5000$. Su Enron la media resta invece relativamente stabile, nell'ordine di 220–230 candidati, pur in presenza di oltre 87 000 nodi.



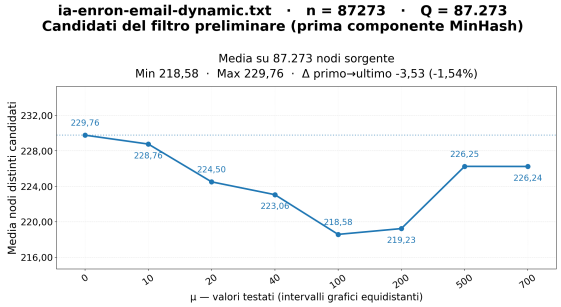
(a) `CollegeMsg`



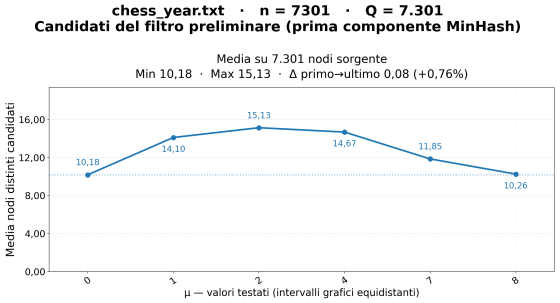
(b) `SFHH-conf-sensor`



(c) `ia-facebook-wall-wosn-dir`



(d) `ia-enron-email-dynamic`



(e) `chess_year`

Figura 13: Numero di nodi candidati distinti restituiti dal filtro preliminare dell'oracolo $\mathcal{O}$ al variare dell'ampiezza temporale μ. Risultati sui dataset: (a) `CollegeMsg`, (b) `SFHH-conf-sensor`, (c) `ia-facebook-wall-wosn-dir`, (d) `ia-enron-email-dynamic` ed (e) `chess_year`.

7 Conclusioni e Sviluppi Futuri

7.1 Conclusioni

Il presente lavoro ha affrontato il problema del Neighborhood Search su grafi temporali, ovvero l'individuazione di nodi con vicinati simili all'interno di intervalli temporali, senza ricorrere a una ricerca esaustiva sull'intero grafo. Partendo dalle tecniche di ricerca per similarità usate sui grafi statici, basate su MinHash e LSH, sono state analizzate due varianti del problema, distinte in base alla conoscenza dell'ampiezza μ durante il preprocessing. Il primo contributo riguarda la variante ad ampiezza fissa, per la quale è stata proposta una soluzione basata su un indice LSH unificato, capace di indicizzare congiuntamente le coppie nodo–tempo. Un elemento centrale è l'insieme $W(v)$, che limita la ricerca alle sole finestre rilevanti per il nodo interrogato. L'analisi teorica ha mostrato che la cardinalità di $W(v)$ dipende dalla distribuzione temporale degli eventi e non solo dal loro numero; gli esperimenti lo confermano, con il 98.2% delle finestre escluse su Enron e il 96.2% su Facebook, ma riduzioni molto più contenute nei dataset con interazioni più dense, come SFHH (10.1%). Un secondo contributo è l'oracolo $\mathcal{O}$ per il recupero degli sketch MinHash dei vicinati temporali. Combinando la componibilità degli sketch con un Range Tree, esso risponde alle interrogazioni su intervalli arbitrari in tempo $O(k \log |\Lambda(v)|)$ con spazio lineare nel numero di archi temporali, evitando la ricostruzione del vicinato a ogni query. Gli esperimenti su grafi sintetici hanno mostrato tempi dell'ordine dei microsecondi anche sui grafi più grandi, contro centinaia di millisecondi della ricostruzione naïve. Il confronto con la Brute Force ha evidenziato riduzioni significative dei tempi di interrogazione, con speedup fino a circa 8431× su Enron e 4287× su Facebook. L'indice unificato si è inoltre dimostrato più veloce in query dell'alternativa basata su $T - \mu$ indici LSH indipendenti (1.33–3.53×), a fronte di un maggiore uso di memoria ausiliaria. Questi vantaggi vanno però letti insieme alla natura probabilistica dell'LSH: la recall rispetto alla Brute Force, calcolata sulle medesime signature, varia tra il 12.17% e il 98.07%, e il filtro può quindi escludere nodi che avrebbero superato la verifica. Anche il beneficio del pruning non è universale: su SFHH la variante senza pruning risulta marginalmente più rapida in query. Il secondo problema affrontato riguarda l'ampiezza variabile, in cui μ è noto solo a query time. Poiché un indice per ogni ampiezza sarebbe troppo oneroso, è stata proposta una soluzione euristica basata sui nodi che condividono la prima componente MinHash in almeno una finestra. La ricerca di tali corrispondenze è stata ricondotta a un problema geometrico di intersezione tra rettangoli, risolto con Range Tree multidimensionali. La valutazione sperimentale si è concentrata sulla selettività del filtro: il numero medio di candidati distinti resta contenuto anche in grafi con decine di migliaia di nodi (circa 220–230 su Enron, con oltre 87 000 nodi). Il risultato suggerisce che la rappresentazione geometrica sia una base promettente per ridurre lo spazio di ricerca. La valutazione riguarda però soltanto la generazione dei candidati e non consente ancora di stabilire prestazioni e completezza dell'algoritmo complessivo, comprensivo della verifica delle signature. Nel complesso, il lavoro mo-

stra come sketch componibili, indicizzazione per similarità e strutture dati temporali possano rendere più efficiente la ricerca di vicinati simili nei grafi temporali. I risultati evidenziano al tempo stesso l'importanza di considerare la struttura temporale dei dati, il costo del preprocessing e il compromesso tra efficienza e qualità delle risposte.

7.2 Sviluppi futuri

I risultati ottenuti suggeriscono diverse direzioni di approfondimento. Una prima direzione riguarda la memoria occupata dalle strutture dati. L'indicizzazione riduce notevolmente il costo delle query, ma introduce un overhead dovuto alla memorizzazione di sketch e riferimenti nelle tabelle hash. Rappresentazioni più compatte, tecniche di compressione e strategie di indicizzazione sparsa potrebbero consentire di trattare dataset temporali ancora più grandi. Una seconda direzione, particolarmente rilevante, è il completamento della valutazione sperimentale della variante ad ampiezza variabile. Gli esperimenti svolti analizzano il numero di candidati prodotti dal filtro preliminare, ma non la successiva verifica con le signature complete. Un passo naturale è implementare e valutare l'intera procedura di interrogazione, misurandone tempi, memoria e recall rispetto a una soluzione esaustiva. Poiché il filtro attuale utilizza una sola componente MinHash, sarebbe inoltre interessante studiare l'impiego di più componenti o di più filtri indipendenti, valutando il compromesso tra maggiore probabilità di individuare nodi simili e maggiore costo di indicizzazione. Un'ulteriore possibilità è l'ottimizzazione delle strutture geometriche usate per la seconda variante: il costo spaziale dei Range Tree multidimensionali rappresenta un potenziale limite di scalabilità, e strutture alternative o rappresentazioni più compatte potrebbero migliorare l'efficienza complessiva dell'indice. Poiché la recall della prima variante dipende fortemente dal dataset, un altro sviluppo consiste nello studio della calibrazione dei parametri b e r rispetto alla soglia τ, per valutare come bilanciare completezza e numero di candidati da verificare. Infine, il lavoro potrebbe essere esteso ai grafi temporali dinamici, in cui nuovi archi e nuovi timestamp vengono aggiunti nel tempo. Le soluzioni proposte assumono infatti che la struttura del grafo sia nota durante il preprocessing; consentire aggiornamenti incrementali dell'oracolo $\mathcal{O}$ e degli indici permetterebbe di applicarle a scenari in cui le interazioni sono generate continuamente. Nel complesso, queste direzioni potrebbero contribuire allo sviluppo di algoritmi di Neighborhood Search temporale con un migliore equilibrio tra accuratezza, tempo di interrogazione, costo di costruzione e occupazione di memoria.

Riferimenti bibliografici

[1] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007. doi: 10.1002/asi.20591.

[2] Benjamin Paul Chamberlain, Josh Levy-Kramer, Clive Humby, and Marc Peter Deisenroth. Real-time community detection in full social networks on a laptop. *PLOS ONE*, 13(1):e0188702, 2018. doi: 10.1371/journal.pone.0188702.

[3] Kamal Berahmand, Sogol Haghani, Mehrdad Rostami, and Yuefeng Li. A new attributed graph clustering by using label propagation in complex networks. *Journal of King Saud University - Computer and Information Sciences*, 34(5): 1869–1883, 2022. doi: 10.1016/j.jksuci.2020.08.013.

[4] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing (STOC '98)*, pages 604–613. ACM, 1998.

[5] Jure Leskovec, Anand Rajaraman, and Jeffrey David Ullman. *Mining of massive data sets. Cambridge University Press*, 2020.

[6] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Similarity search in high dimensions via hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*, pages 518–529. Morgan Kaufmann Publishers Inc., 1999.

[7] Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of the Compression and Complexity of SEQUENCES 1997*, pages 21–29. IEEE Computer Society, 1997.

[8] Andrei Z. Broder, Moses Charikar, Alan M. Frieze, and Michael Mitzenmacher. Min-wise independent permutations. *Journal of Computer and System Sciences*, 60(3):630–659, 2000.

[9] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3): 97–125, 2012.

[10] Othon Michail. An introduction to temporal graphs: An algorithmic perspective. *Computer Science Review*, 19:60–78, 2016.

[11] Arnaud Casteigts, Paola Flocchini, Walter Quattrociocchi, and Nicola Santoro. Time-varying graphs and dynamic networks. *International Journal of Foundations of Computer Science*, 23(5):915–948, 2012.

[12] Matthieu Latapy, Tiphaine Viard, and Clémence Magnien. Stream graphs and link streams for modeling interactions through time. *Theoretical Computer Science*, 722:28–68, 2018.

[13] Jon Louis Bentley. Decomposable searching problems. *Information Processing Letters*, 8(5):244–251, 1979.

[14] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*. Springer-Verlag, Berlin, Heidelberg, 3rd edition, 2008.

[15] Michael A. Bender and Martín Farach-Colton. The LCA problem revisited. In *LATIN 2000: Theoretical Informatics, 4th Latin American Symposium*, volume 1776 of *Lecture Notes in Computer Science*, pages 88–94. Springer, 2000.

[16] Johannes Fischer and Volker Heun. Theoretical and practical improvements on the RMQ-problem, with applications to LCA and LCE. In *Combinatorial Pattern Matching, 17th Annual Symposium (CPM 2006)*, volume 4009 of *Lecture Notes in Computer Science*, pages 36–48. Springer, 2006.

[17] Eduard Helly. Über mengen konvexer körper mit gemeinschaftlichen punkten. *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 32:175–176, 1923.

[18] Dan E. Willard. New data structures for orthogonal range queries. *SIAM Journal on Computing*, 14(1):232–253, 1985.

[19] Bernard Chazelle and Leonidas J. Guibas. Fractional cascading: I. A data structuring technique. *Algorithmica*, 1(2):133–162, 1986.